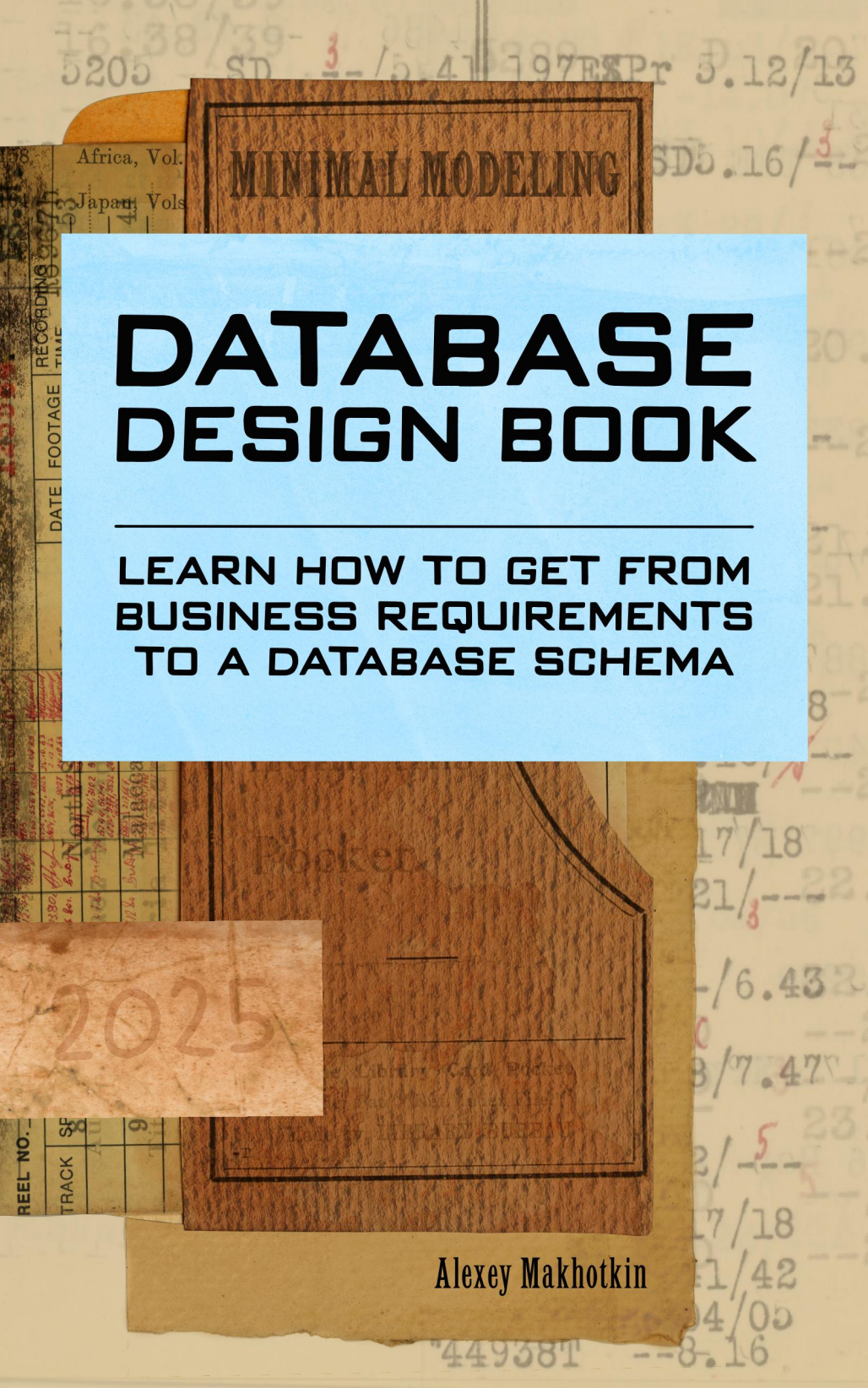


DATABASE DESIGN BOOK

LEARN HOW TO GET FROM BUSINESS REQUIREMENTS TO A DATABASE SCHEMA

Alexey Makhotkin

DATABASE DESIGN BOOK

learn how to get from business requirements to a database schema

ALEXEY MAKHOTKIN

2025

First Edition (revision: 2025-05-17)

Database Design Book: https://databasedesignbook.com/

Minimal Modeling Substack: https://minimalmodeling.substack.com/

Twitter: https://twitter.com/alexeymakhotkin

Cover design by Anna Golde
(https://www.linkedin.com/in/annagolde/).

CONTENTS

1

INTRODUCTION

This is a book on database design. You can find book updates and extra material at https://kb.databasedesignbook.com/.

1.1 WHO IS THIS BOOK FOR?

The goal of this book is to help you get from a vague idea of what you need to implement (e.g., *"I need to build a website to manage schedule and instructor appointments for our gym"*) **to the comprehensive definition of database tables.**

If you're a beginner, this book is for you. If you know that you need a database for your project, but are not sure how to begin, this is for you, too. If you've learned something about database design theory, but not sure if your design would work: you'll find help here. And if you struggle with some parts of the business domain, while the rest is clear, the approach laid out in this book will give clarity.

1.2 WHAT LEVEL OF UNDERSTANDING IS THIS BOOK AIMED AT?

The first part of the book is completely database-agnostic. All you need to understand is how your business works. This does not have to be a business, either: just some problem that necessitates a database.

The second part of the book requires some level of familiarity with common databases: how the tables are created, what physical data types exist, what are primary key and index, how the tables would be queried, and how to insert and update data.

We begin by assuming that you will use one of the traditional relational database servers, such as MySQL or PostgreSQL.

You do not need to understand database theory, such as normal forms.

1.3 WHAT YOU WOULD GET FROM READING THIS BOOK

The idea of the book is to make every step of table design actionable.

You will learn **how to extract the logical model** from a free-form description of the problem. This will also help to clarify your understanding of the problem, because the process forces you to be more precise and reduces hand waving.

You will learn **how the logical model is translated into a physical table design**. There are a dozen possible ways to represent logical models in physical space. First we discuss one such design strategy (a table per anchor); later we'll see why one might want to occasionally use a different one .

The book will help you answer a common question: **how can I prove, even to myself, that my design is complete**? Also, you will be able to defend this design, be it at a job interview or in a database exam.

A logical model is **a very good way to explain your design to other people**, for example to your team members or business stakeholders. This way of presenting your design immediately answers many questions that people may have. Because of that they can give better feedback and confirm that you're on the same page.

You will learn to **think separately about logical and physical aspects of your database**. This brings a lot of clarity: your problem lies either on the one or the other side, and you can save a lot of complexity if the other side is fixed.

Our approach is strongly rooted in relational modeling. However, it is **designed to accommodate non-relational NoSQL systems**, and even more specialized technologies such as Amazon S3. As mentioned earlier, the logical model is not dependent on any specific database server system: you just have to choose a suitable table design strategy.

1.4 WHO I AM

My name is Alexey Makhotkin. I have been working with databases for more than 25 years in various roles: as a software engineer, database administrator, team lead, head of software engineering. I've built and helped to build dozens of schemas over the years.

In 2021 I started the "Minimal Modeling" substack: https://minimalmodeling.substack.com/, trying to summarize what I have learned. This book is another step. You can contact me at squadette@gmail.com.

2

BUILDING A LOGICAL MODEL

Suppose that you want to **build a software system that stores some data in a database**. It may be one of hundreds possible kinds of software systems, such as:

- ecommerce: placing and fulfilling orders;
- money-tracking software, such as personal budget tracker or a business accounting system;
- social network with users, posts, and comments;
- tracking your progress and achievements in a role-playing game;
- etc., etc.

In each of those systems there is some sort of underlying database, with tables, columns, rows and datatypes. Your task is to design and build this database. Then the system will use it to store and retrieve the data.

How do you design and build this database? This chapter, as well as the entire book, aims to answer exactly this question.

2.1 SYSTEM REQUIREMENTS AS INPUT

What exactly are we building? In this book, it is your responsibility to specify the system that you are building. You need to gather and provide answers about what the system is supposed to do.

The point of this chapter is that your understanding of the requirements will gradually evolve.

In the beginning you have just **an informal text that roughly specifies what the system does**. Imagine that you sit down with somebody and explain what the system is supposed to do. They ask clarifying questions, you respond, and so it goes back and

forth for some time, maybe for an hour or two. Suppose that your discussion was recorded, and someone transcribed and wrote it down as an informal specification.

Then we're going to build **a logical data model** based on this informal specification, using the approach presented in this book. As a result, you will have a structured document in a tabular format. In this chapter, we discuss how the logical model is built and validated.

Based on the logical model, we're going to **design the physical database**. This is to be covered in the next chapter, "Building the physical schema".

2.2 WHO DO WE WRITE THE LOGICAL MODEL FOR?

When the logical model is written down, who is going to read it, and why?

This book is aimed at three scenarios:

- you, the reader, want to **learn database design** by implementing the entire project yourself;
- you, the business analyst, want to **describe the system for other people to implement**;
- you, the project group leader, want **to make sure that everyone in the group is on the same page**.

2.2.1 *For yourself*

If you want to learn database design, the main goal of the logical model is to organize your own thoughts. If you're new to database design you may just be overwhelmed, and the approach explained in this book can help.

Decoupling the logical phase from the physical phase of database design greatly reduces complexity of the process. Working on the logical part, all you need to think about is the nature of the problem: your business requirements. When the logical part is figured out, you can switch to thinking about the physical design: tables, indexes, data types, queries, cached data, etc.

In both phases, a lot of thinking capacity is freed by not having to worry about the other phase.

I expect that you will soon outgrow this book, like the training wheels on a child's bike. But as your skill in database modeling and design grows, something entirely different happens as well. Your systems become more complex, and more people need to work on those systems. It means that you have to explain the system to different people who have different concerns, needs, and skill levels.

2.2.2 *For software developers*

You may find yourself in the role of a business analyst. Someone else is going to actually build the system (implement the database, write code, etc.). But your job is to understand the problem and to refine business requirements.

You can use this approach to write down the logical design to validate your understanding of the business requirements. You confirm the requirements with business stakeholders, but you don't need to think about the physical implementation: just leave the corresponding columns blank.

When you pass the logical model to the implementing team, you can be sure that all the database elements are clearly presented to them. They immediately know the descriptions, data types, they can see example values. They can cross-check their own understanding of the business requirements with the logical model, and confirm mutual understanding.

2.2.3 *For the project group*

As a project group leader, you may find our approach useful for making sure that everyone is on the same page.

The logical model is simple and straightforward. It is designed to be collaborative, because anyone can update the shared logical model document. It has very low requirements for the level of experience in design and business analysis.

2.3 CAN I SKIP THE LOGICAL MODEL?

A logical model exists even if you refuse to acknowledge it.

If you were to skip the logical model and begin working on a physical schema straight away, you would have to handle two different concerns simultaneously. One concern is your business requirements: how the system is supposed to work. Another concern is the physical schema organization: how the data is going to be stored in an optimal way. An explicit logical model simplifies both parts, because you can think through the logical model before you attend to the physical implementation details.

When the logical model is not written down explicitly, it is scattered in three places: physical database schema, system code and people's memory. People new to the system must pick up pieces of the logical model from schema, code and other people. Unfortunately, this process can give ambiguous results. This happens even in the most common situations, such as asking for the database design advice. It's hard to give meaningful advice without a clear explanation of the underlying problem.

2.4 CAN I USE AN ERD (ENTITY-RELATIONSHIP DIAGRAM) INSTEAD?

There are several well-known graphical notations for databases. There is UML, ERD (Entity-Relationship Diagrams), IDEF1X, and some others. On those diagrams you can see some rectangles connected by lines, with attributes and such attached to the rectangles. There are many different styles of notation, particularly how the lines between the rectangles are represented.

To draw a diagram, you have to do all the work of defining a logical model. Then you represent it graphically, using a standard notation of your choice.

In this book, we're still going to use some graphical representations to help with understanding the system. But the tabular form would be the primary format for the logical model as explained here.

2.5 ELEMENTS OF A LOGICAL MODEL

Let's introduce the three main concepts of a logical model:

- anchors;
- attributes;
- links.

Anchors are nouns, entities, objects. "*User*" and "*Order*" are common examples of anchors.

Attributes contain the actual information about anchors. "*Name of the User*" is an attribute. "*Item price*" is another attribute.

Links connect two anchors, or establish a relationship between them. "*User has placed an Order*" is an example of a link.

We'll describe all three in great detail in this chapter. Every database could be represented as a combination of anchors, attributes, and links. We will show how to get from the business requirements (informal text) to a logical model (a structured list of anchors, attributes, and links).

2.6 THE PROCESS

But just understanding the definitions is not enough to build a useful model: we also need to describe the process.

People often present the draft of their database schema and ask a very common question, "How can I make sure my database schema is correct?"

Sometimes one needs to present a graphical schema as a logical model. Similar questions arise, "Is my graphical schema right? Do you have any feedback on my logical design?"

One of the stated goals of our process is that to a large extent, such questions would be answered automatically.

The process is iterative and incremental.

The overall process goes like this (this is a major peek-ahead, we'll discuss the entire process in this chapter):

Step 1. Begin with the **business requirements** (written down, or maybe even just listed in your head);

Step 2. Find **anchors** that you see in the requirements, write them down;

Step 3. Find **attributes** that belong to the anchors, based on the requirements, write them down;
Step 4. Occasionally, here you may realize that you have **missed some anchors**, go back to step 2;
Step 5. Find **links** that connect the anchors you've found so far;
Step 6. Again, sometimes here you may discover that you have **missed some anchors**, go back to step 2;
Step 7. Go back to the business requirements and **highlight all the database elements** that you have found so far; if any words are not covered, go back to steps 2, 3, or 5;
Step 8. Now that the logical model makes sense, we can go over it and translate it into a physical schema: write down all the tables, columns and physical data types. That's it.

2.7 ANCHORS: INTRODUCTION

Let's just begin with some examples: *User*, *Order*, *Item*, *Post*, *Comment* are anchors. As you can see, anchors are nouns.

Anchors are anything that could be counted, one by one. For example, you can say: *"Here is our first user, here is our second user, etc. We have 1000 users total in our database."*

Anchors are only concerned with IDs, they do not contain any data/information. The information is stored in attributes, as discussed below.

You can find most anchors by reading the system requirements description and looking for nouns. Not every noun is an anchor, though. We have to confirm that by using two formalized validation sentences: counting sentences and adding sentences.

2.7.1 *Example: posts*

Let's use "*Post*" as an example. Suppose that our requirements contain this sentence, "Users can publish posts". One anchor here is "*User*", another one seems to be "*Post*". Let's validate if the latter is actually an anchor by using the sentences.

The first validation sentence is, "*We have 1000 Posts in our database so far*". This sentence makes perfect sense.

The second validation sentence is, "<*When you click a button*>, *the system inserts another Post in the database*". This sentence also makes sense: that's how we confirm that this is really an anchor.

If one or both sentences sound awkward or outright incorrect, the noun is most probably not an anchor, but refers to an attribute or a link. It is really helpful to say those sentences out loud, as if you were explaining your system to another person.

2.7.2 *A list of anchors*

We write down **the list of anchors** using the following template (two example anchors are shown here).

Anchor	*ID example*	*Physical table name*
Post	1, 2, 3, ...	`posts`
Comment	1, 2, 3, ...	`comments`

We need to define just three things about each anchor: its name, its IDs, and its physical representation.

Names are very simple and straightforward for most simple anchors, but may be challenging in some cases. We'll talk about that later in the "Advanced anchors" section.

Most of the time **anchor IDs** would be just some numbers without any specific meaning. You can have, e.g., User with ID=1, User with ID=2, etc. But sometimes IDs would look differently: see below.

We will talk about **physical tables** in the second part of the book, "Building physical schema".

It's not necessary to have a complete list of anchors before proceeding with attributes and links. If you work on attributes or links and find an anchor that you've missed, you can always go back and add it to the list.

2.7.3 *Example: invoices*

Many anchors are simple and straightforward. You can point at a thing in the real world, such as a restaurant or a ticket, and it's going to be an anchor.

Some anchors are a bit more complicated. Let's take a look at invoices. An invoice itself is a simple and straightforward anchor. But every invoice contains an itemized list of services rendered, or items sold. Here is an example of an invoice:

Invoice #203 (Date: 2024-01-05)	
Services rendered	Cost, USD
Pruning branches	120.00
Mowing grass	50.00
Fixing the fence	40.00
Total	210.00

One can count invoices (*"We sent out 200 invoices so far"*). But we can also count invoice items and add new items. For example, we have a multi-day project and keep track of the services rendered by adding new items to the draft invoice.

Let's write down the validating sentences for the *InvoiceItem* anchor:

- "*We have delivered 750 items of work to our customers*";
- "*Clicking this button adds another line item to the opened invoice*".

Later in the "Links" section we'll discuss how *Invoice* and *InvoiceItem* anchors are connected. Here is how the list of anchors for this example looks like:

Anchor	*ID example*	*Physical table name*
Invoice	1, 2, 3, ...	`invoices`
InvoiceItem	1, 2, 3, ...	`invoice_items`

Additionally, there is another very common pattern you must understand: *the repeated sales pattern*. It covers things like cinema tickets, flight tickets, restaurant reservations, etc. We'll discuss this pattern in the "Repeated sales pattern" chapter.

In this section we've learned that:

- anchors are nouns;

- anchors serve as a foundation for the logical model, because attributes and links depend on the anchors;
- to confirm that an anchor is valid, use two validating sentences:
  - a counting sentence: "There are X posts in our database";
  - and an adding sentence: "Click this button to create a new post";
- a list of anchors is presented in a tabular format with three columns: name, ID example, and physical storage information;
- to find most of the anchors, go over business requirements, find all the nouns and validate them.

2.7.4 *Anchor IDs*

Anchors are concerned with only one thing: keeping track of IDs to support attributes and links. Anchors themselves do not contain any information, attributes do. (Attributes are discussed in the later sections.)

Attributes depend on anchors: they use those IDs to store the actual data. Links also depend on anchors: they connect two different IDs, storing the relationship data.

For now we'll introduce the simplest possible approach to IDs. Let's assume that IDs are just sequential integer numbers: 1, 2, 3, 4, 5, ...

So, for example you have an Order with ID=1, an Order with ID=2, an Order with ID=3, and so on.

Other anchors have their own IDs. In the same database, you can have a User with ID=1, a User with ID=2, and so on.

This is enough to begin with. But there is a lot of depth around IDs, and we're going to discuss them further in two separate sections. Logical aspects of IDs would be covered in the "More on IDs and unique attributes" section later in this chapter. Physical aspects of IDs would be covered in "Anchors: physical IDs" in the "Building a physical schema" chapter. See also "Well-known anchors" for some interesting examples of anchor IDs (currencies, languages, countries, and days of the week).

2.8 ATTRIBUTES: INTRODUCTION

Attributes contain the actual information. Here are some informal examples of the attributes:

- title of the Book;
- User's date of birth;
- expiration date of the Gift Card;
- Is this Restaurant pet-friendly?
- Item's price;
- etc., etc.

You can see that each attribute mentions an anchor: *Book*, *User*, *GiftCard*, *Restaurant*, *Item*.

In the previous section we learned that anchors only manage IDs. Attributes contain some information about their anchors: they store a small piece of data for each ID.

Let's take *"User's date of birth"* as an example. We can store the following information about some users:

- *A User with ID=20 was born on 1923-08-19;*
- *A User with ID=23 was born on 1941-01-18;*
- etc., etc.

2.8.1 *Attributes: definition*

To describe each attribute, we need to know six things about it:

- Its anchor;
- A human-readable question;
- Its logical data type;
- An example value (for humans);
- Its physical column name;
- Its physical data type;

In the first part of the book we're going to talk about the first four things. Later we'll discuss the physical design of attributes.

Let's present some examples of attributes, using a tabular definition format that we're going to use.

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Book	What is the title of the Book?		
string	*"On the Origin of Species"*	⟨TBD⟩	
Book	How many page numbers are in the Book?		
integer	*230*	⟨TBD⟩	
Item	What is the price of this Item?		
monetary amount	*19.99*	⟨TBD⟩	
User	What is the date of birth of this User?		
date	*1947-09-21*	⟨TBD⟩	
Room	Is this room wheelchair-accessible?		
yes/no	*yes*	⟨TBD⟩	
Post	When is this post to be published? (in UTC timezone)		
date/time	*2023-08-10 09:30:00*	⟨TBD⟩	
Ride	What is the status of the Ride?		
enum	*"requested"* *"driver_chosen"* *"arriving"* *"in_progress"* *"completed"* *"canceled"*		
Country	What is the ISO code of this Country?		
string	*"pt"*	⟨TBD⟩	

2.8.2 *Attributes and anchors*

The first column refers to the attribute's anchor. Those anchors must be listed in the table discussed in the "Anchors" section. For many attributes the anchor is obvious.

But sometimes, in more complicated cases, as you try to describe an attribute, you may realize that you have missed an anchor. In that case you can return to the list of anchors and just add one.

2.8.3 *Human-readable questions*

To describe what the attribute contains, we use human-readable questions in a somewhat formalized format.

In informal communication, people talk about attributes using something like *"User's date of birth"*, *"Post publication time"*, *"Book title"*. For many simple attributes, this may be enough to understand the meaning of the attribute.

We use questions because they handle more complicated cases, where precise definitions become more important.

One reason why we choose questions is because some of the attributes require the question form: for example, "*Does this user have administrator rights?*" and "*Is this To-do item done?*".

The second reason is that questions offer more space for clarification. Let's take "item price" as an example. We need to know if that's a price with or without a tax, or with or without a discount.

Longer sentences also help people to confirm that the definition of the attribute makes sense. You can read the question out loud and sometimes notice that the definition is not precise.

2.8.4 *Example values*

Example values of the attribute are invaluable for the understanding of the new system. Sometimes even the human-readable question may not be enough, just because you do not understand the business domain sufficiently.

Looking at a good sample value, or several values, helps to reconfirm your understanding or shows that your initial assumption was incorrect.

One important distinction for string attributes is whether the string is human-readable or machine-readable. This is also something that you would see right from the example value.

2.8.5 *Data types and types of data*

Now, let's discuss what kind of data can be contained in an attribute. In the previous section we saw many examples of data attributes, which could be classified in the following groups:

- **strings** (e.g. SKU, filename, names of all kinds, etc.);
- **integer numbers** (image width, number of bedrooms, etc.)
- **monetary amounts**;
- **numeric values with a fractional part** (geo coordinates, temperature, weight, etc.)
- **"yes" or "no"** ("*is To-do item done?*", "*is the Room wheelchair-accessible?*", etc.). This is also called "boolean values", but we avoid using that term here.
- **either/or/or values** (also called **enums**). The value for this attribute can be from a strictly defined list of values. (For example, let's consider the "*What is the status of this Ride?*" attribute. It may have one of the following values: *"requested"*, *"driver_chosen"*, *"arriving"*, *"in_progress"*, *"completed"*, and *"canceled"*.) This is discussed in a separate section.
- **dates:** day of birth, contract start date, etc.
- **date with time**: when something happened, or when something is scheduled to happen. You must specify the associated timezone (UTC or local): see the section below. This time can be up to a second, or even a fraction of a second.
- **binary blobs**, such as JPEG file (this is a special case, we'll discuss it in a separate section);

We can see here that attribute values are simple, there is only a single value.

2.8.6 *What is not an attribute?*

Some things are NOT attributes, or require more careful handling.

Attributes cannot contain anchor IDs. "*Who is the manager of this Employee?*" is wrong. This case is modeled as a link (see the "One-to-many links" section below).

JSON belongs to the physical model. Many modern databases support JSON-typed columns. JSON allows you to store a lot of data in a single column. But you cannot have a JSON-typed *attribute*. Instead, first you need to specify all the attributes and links as normal. See the "JSON table design strategy" below chapter for more discussion.

2.8.7 *How to confirm that all the attributes have been listed?*

How many attributes would we have? A typical number of attributes per anchor is around half a dozen, sometimes less, sometimes more. Two is also okay. In some systems some anchors may have a lot of attributes, up to a few hundreds. This is often the result of many years of system growth. For the purposes of this book, expect to have three to five times the number of anchors on your list.

To confirm that you have listed all the attributes, go over the original business requirements. Find all the fragments that mention a piece of information that needs to be stored. Make sure that this piece of information was added to the list of attributes and highlight those fragments. That way you can find missing attributes or confirm that none are.

Sometimes you may discover that you've missed an attribute only when you actually begin implementing the system. You can just go back to the list of attributes and add it there.

Why do we want to have a more or less complete logical model? Because we want to reduce project risk. We think the system through while it's still a document that can be edited cheaply. Later, as the system gets implemented, fixing the incorrect data model may turn out to be much more costly.

Now that we have a list of anchors and attributes, let's work on links, the last part of the triad.

2.9 LINKS: INTRODUCTION

Links connect two anchors. Here are some examples:

- *A Customer placed an Order;*
- *An Attachment is attached to a Message;*
- *An Employee is the manager of an Employee;*
- *A Post was written by a User;*
- *A Post was liked by a User;*
- *A Post was bookmarked by a User;*
- *A Flight is operated by an Airline;*

As you remember, an anchor's primary job is to keep track of IDs. Links keep track of pairs of IDs. For example, let's take "*A Customer placed an Order*". We would have the following sample of records in our database:

- *A Customer with ID=2 placed an Order ID=210;*
- *A Customer with ID=17 placed an Order ID=211;*
- *A Customer with ID=3 placed an Order ID=212;*
- *A Customer with ID=2 placed an Order ID=220;*
- *etc., etc.*

2.9.1 *Links: definition*

To describe a link, we need three logical pieces of information:

1. two anchors that are connected by this link;
2. two sentences that describe the business role of the link;
3. the link's cardinality (one-to-many, many-to-many, one-to-one);

and a physical descriptor:

4. A table or column that stores the ID pairs;

We're going to discuss the first three in the first half of the book, and the physical storage in the second half. Here are three examples of typical links. We'll consider each element of this table in detail shortly.

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Customer : Order	**1:N**	A Customer places *several* Orders An Order can be placed by *only one* Customer	⟨TBD⟩
Employee : Employee	**1:N**	An Employee is the manager of *several* Employees An Employee has *only one* Manager	⟨TBD⟩
Post : User	**M:N**	A Post can be liked by *several* Users A User can like *several* Posts	⟨TBD⟩

Correct link specification is crucial for the database design that corresponds to the business requirements. This table is optimized for an error-proof specification of link structure. To achieve this goal, in this table we introduce quite a lot of redundancy. As you will see, in each row, a pair of anchors is mentioned no fewer than three times. Cardinality is also specified twice.

Links may be the most interesting element as compared to anchors and attributes. Every link is a fragment of some business process that is served by the database that we're designing.

2.9.2 *Links: pair of anchors*

Every link connects two anchors. It could be two different anchors or just one anchor that connects to itself. One of the most common examples of the same anchor on both sides is: "*Employee is the manager of Employee*".

Between two different anchors, there could be several different links using different verbs. For example, you could define at least three links connecting Posts and Users:

- *A Post was written by a User;*
- *A Post was liked by a User;*
- *A Post was bookmarked by a User.*

2.9.3 *Links: cardinality*

Let's take the link that connects *Customer* and *Order* as the first example. Suppose that there is a specific customer with ID=3. How many orders can a certain customer place? It's clear that there could be **several** orders; for example, this customer placed four orders with the following IDs: 210, 211, 212, 220.

Now let's look at one of the orders, say ID=211. How many customers can place a certain order? In virtually all e-commerce systems **only one** customer can place an order. (In this case, it's the customer with ID=3.)

Note that we've bolded two fragments: "**several** (orders)" and "**only one** (customer)". This means that the cardinality of this link is "many-to-one". (We'll discuss the list of possible cardinalities below.)

Here is a second link example: *Projects* and *Developers*. Suppose that we implement a project tracking system that allows us to assign developers to projects. How many developers could be assigned to a certain project? It's clear that there could be **several** developers.

Now let's take a look at one of the developers. How many projects could be assigned to a specific developer? In virtually all project management systems you can assign **several** projects.

Note again that we've highlighted two fragments: "**several** (developers)" and "**several** (projects)". This means that the cardinality of this link is "many-to-many".

At the logical level, three cardinalities are possible:

- many-to-many (M:N);
- one-to-many (1:N);
- one-to-one (1:1).

Note that "many" here also includes the case of "zero items". For example, a newly-registered customer has no orders placed yet.

2.9.4 *Cardinality is a business concern*

Correct specification of link cardinality is super important. Physical design crucially depends on it. Changing cardinality is possible, but this is perhaps the most complicated procedure as compared to other types of changes in a typical database. So it's important to validate it, and that is why we use human-readable sentences.

Note that only you are responsible for specifying the correct link cardinality. You may discuss this with stakeholders on the business side and they will tell you how the business operates. If you're the business stakeholder yourself, you know what cardinality you want. The point is that no one from outside the project can "tell" you what the cardinality of a certain link is. Particularly, it makes no sense putting this question to the LLM.

Here is an example of how to make different decisions for the cardinality of a link. Suppose that you have a blog with Posts and Categories. We have a link: "*Post belongs to a Category*". Let's write down the sentences:

1. "A *Category may contain **several** Posts."*

This part is clear. However, we're less certain about the second sentence:

2a. "*A Post can belong to **only one** Category*."
2b. "*A Post can belong to **several** Categories*."

Which of the two is correct? This is your call: it's a question of how your product works. You may want to make it simple ("*only one Category*") or more flexible ("*several Categories*").

What if you change your mind when you already have a website running? You started with one Category, but now you think that you really need several. Changing the cardinality of a link is not trivial, but it's not super hard, either. We'll discuss that later in the "Evolving your database" chapter.

So, this is something that you must decide beforehand: it's going to be just as you specified it.

2.9.5 *Links: sentences*

To validate the link definition, we use a pair of sentences for each link. Sentences are supposed to be human-readable (even though they may sound a bit awkward at times).

Here is an example:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Customer : Order	**1:N**	A Customer places *several* Orders An Order can be placed by *only one* Customer	⟨TBD⟩

It's often impossible to say which of the two anchors is more important than the other one. In other words, which is a subject and which is an object. Because of that we made a decision to use both.

In both sentences we must use either "*several*" or "*only one*". Why not *"many"* and *"one"*? In our experience, "several" and "only one" sound more natural, so we use them in all examples for consistency.

However, we still use "*many-to-many*" and "*many-to-one*" in the "Cardinality" column, and not "*several-to-only-one*" and "*several-to-several*", for alignment with common usage (and also because it's faster to say).

A good way to confirm the correctness of a link definition is to read it aloud (to someone else or to yourself). You can do that even with people far removed from the technical side of the project: business domain experts and other stakeholders. If the sentence doesn't sound right, they'll tell you, and you'll have the chance to fix it or to discuss why your understanding is different. In our experience, sometimes just reading the modeling sentences out loud helps people to understand the data flow.

2.9.6 *False links and unique pairs of IDs*

You may remember that anchor IDs are unique. Links correspond to a list of pairs of IDs, one for each anchor. It's important to understand that those pairs are unique.

Let's go back to our example of users placing orders:

- *Customer ID=2 placed an Order ID=210;*
- *Customer ID=17 placed an Order ID=211;*
- *Customer ID=3 placed an Order ID=212;*
- *Customer ID=2 placed an Order ID=220;*
- *etc., etc.*

Let's represent this in a shorter tabular form:

Customer ID	*Order ID*
2	210
17	211
3	212
2	220
...	...

Note that each pair of IDs is unique here. This is because it makes no sense for a customer to place the same order twice.

Understanding this can help to prevent modeling mistakes in some scenarios. Here is an example: suppose that we have a simple e-commerce system that has Customers and Items. Can we say "A *Customer buys several Items*" and "*An item could be bought by several Customers*"?

Suppose that an apple pie has ID=1001, and a cup of coffee has ID=1020. Consider this (incorrect) table that holds customer purchases. Suppose that a Customer with ID=3 buys a cup of coffee, and a Customer with ID=8 buys a coffee and an apple pie:

Customer ID	*Item ID*
3	1020
8	1001
8	1020

What if the first customer wanted to buy a cup of coffee again the next day? The table as it is would not be able to reflect that second purchase, as it cannot contain two identical pairs of IDs.

That's why the correct way to model this is to introduce an "Order" anchor and use two links:

- *A Customer can place several Orders*; *An Order can be placed by only one Customer*;
- *An Order can include several Items*; *An item can be included in several Orders*.

We call such modeling mistakes "false links". They occur because the sentence *"A Customer buys an Item"* sounds so natural! But when the uniqueness criterion is applied, you will see that you've missed something (in that case, an Order anchor).

2.10 MORE ON ANCHORS

Earlier in this chapter we introduced the simplest anchor IDs: sequential integer numbers. In this section we'll introduce several real-world use cases that require some other approaches to identifying things.

2.10.1 *Unique attributes*

Anchor IDs directly correspond to real-world things. If you have 10 users registered in your database, you're going to have exactly 10 anchor IDs: numbers from 1 to 10.

Suppose that we need to implement a tiny CMS (content management system). We want to create and edit pages for our website. Let's write down a very simple logical model. First, we have the "Page" anchor:

Anchor	*ID example*	*Physical table name*
Page	1, 2, 3, ...	⟨TBD⟩

Let's define an attribute that stores the text on the page:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Page	What is the **text** of this Page?		
string	*"Our company was established in 1983..."*	⟨TBD⟩	

You may want to add a few more attributes, but that's enough for our current needs. Each page corresponds to a website URL. But what does the URL look like? We only have an integer number, so we can build only page URLs that look like this:

```
https://example.org/content?page_id=3
```

When a customer visits this page, our application would take the "`page_id`" parameter, query the database using the ID value (in this case, "3"), and show the corresponding text to the customer. But what if we want human-readable URLs, such as

```
https://example.org/about
```

Let's introduce another attribute!

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Page	What is the **URL slug** of this Page?		
string, unique	*"/about"*	⟨TBD⟩	

Now we can change our application so that it looks at the URL, extracts the "`/about`" part, finds the requested page by the attribute value, and shows the corresponding text to the customer. (Note that the previous URL scheme can coexist with the new one.)

If you look at the "Data type" column in the table above, you'll see that it says "string, unique". This is a logical data type. We'll

discuss the implementation of uniqueness in the "Building a physical schema" chapter.

2.10.2 *Optional unique attributes*

We're free to design our CMS as we want. What if we want to allow that some pages do not have a specific slug? For example, we want to have draft pages whose URLs have not been decided yet. Or maybe the page could only be accessible as "`/content?page_id=3`", and we are fine with that.

We'll discuss the implementation of this scenario later in the "Building a physical schema" chapter.

But this case shows an important difference between anchor IDs and unique attributes. It's impossible to have an anchor (Page, in this example) without a specific ID. But it's possible to define a unique attribute that sometimes has no value set.

So, each unique attribute value corresponds to a thing, but not every thing has a corresponding attribute value. In contrast, each anchor ID corresponds to a thing, and every thing has a corresponding anchor ID.

This all sounds very formal, but it is important to keep in mind for some use cases.

2.10.3 *External IDs*

Suppose that we implement an e-commerce system. Naturally, we're going to have the Order anchor. It will have the usual anchor IDs: 1, 2, 3, etc.

How are we going to identify orders for the customer? When a customer places an order, we send them an email saying, *"Thank you for your order #1234! Should you have any problems with that order, please contact our customer support and provide the order number mentioned above"*. Would that be enough?

There are two potential issues with this implementation: industrial espionage and ID enumeration. If order numbers are increasing monotonically, our competitors can monitor our business performance. They can make a small order each day and see exactly how many orders we have received in the preceding 24 hours. This may be undesirable.

To prevent this from happening, one may want to introduce a separate sequence of IDs that is less predictable. For example, as one possible strategy, one can increase the visible ID by a random amount.

The anchor ID stays the same, but it is only used internally and is not visible to customers. Visible IDs would be defined similarly to the "URL slug" attribute above:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Order	What is the customer-visible Order **number**?		
string, unique	*"2024/05067"*	⟨TBD⟩	

2.10.4 *External ID enumeration*

Another related problem with monotonically increasing IDs is that you can try each of them one by one. Consider a system that sells plane tickets. Systems like that often do not require you to create an account. When you buy a ticket, they want to give you access to this ticket. Of course, they do not want strangers to be able to guess some order number and see the details of somebody's flight.

One common strategy is to generate a large random value, for example, using the UUID (Universally Unique Identifier) approach. Then we use this value in the URL, and send this URL to the customer's email. For example, a ticket would be available at:

`https://sample-airline.com/ticket-`**`6D4E900E-162F-40FE-AC02-1A6257552736`**`.pdf`

The part in bold is the value of "ticket UUID" attribute.

2.10.5 *Several unique IDs*

Let's extend our CMS example a bit further. Sometimes we may want to change the URL of a certain page (for search engine optimization), but we also want to keep the old URLs so that they do not break. For example, we want to change "`/about`" to "`/company`", but when the user visits "`/about`", it redirects to "`/company`". Furthermore, we want to be able to keep several such legacy URLs.

Let's build a logical model for this. We used the word "several", so it means that we need a new anchor:

Anchor	*ID example*	*Physical table name*
LegacyURL	1, 2, 3, ...	⟨TBD⟩

We keep the URL slug in an attribute:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
LegacyURL	What is the **slug** of this LegacyURL?		
string, unique	*"/our-company"*	⟨TBD⟩	

Now we need to associate legacy URLs with pages:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Page : LegacyURL	**1:N**	A Legacy URL belongs to *only one* Page A Page can have *several* LegacyURLs	⟨TBD⟩

The main URL of the Page is still recorded as the "Page URL" attribute as before.

We will talk about the physical design of this case in a later section: there are some interesting questions regarding the IDs.

One thing that we notice here is that again, as in the "Optional unique attributes" section, you can say that:

- for each unique identifier (URL slug) there is one thing (Page);
- the opposite is not true: a Page can have no URLs, one URL, or many URLs.

That's, again, as opposed to the anchor IDs: each anchor ID corresponds to a certain Page, and each Page has its anchor ID.

2.10.6 *Implementing physical IDs*

We discussed several real-world use cases that require identifying things from their human-readable IDs. Note that this discussion never mentioned any physical concepts, such as tables, columns, indexes, physical data types etc. We're going to discuss that in a separate section later in the "Building a physical schema" chapter.

2.11 HANDLING TIME IN THE LOGICAL MODEL

Let's talk about date/times: values that look like "*2024-11-19 23:37:00*" in the ISO 8601 format. To understand when this moment of time actually happens, we must specify the timezone. Basically, value without a timezone is meaningless.

There are three main scenarios, in which a timestamp would be used:

- when something actually happened;
- when something will happen N seconds from now;
- when something will happen at a specific time in a given location.

Let's discuss each case separately.

The first case records when **something actually took place**. For example, a customer placed an order at "*2024-11-19 23:45:02*",

Amsterdam time. How do we store this information in the database?

The correct answer in that case is virtually always to use UTC (Coordinated Universal Time). Do NOT store local time directly.

Even if your business only operates in a single country, you should really just use UTC for timestamps of this sort. The number of people who neglected this advice and later regretted this is constantly growing, and this is your chance to avoid this pretty silly and preventable mistake.

The second case is when **something will happen N seconds (or minutes) from now**. For example, a customer requested some sort of SMS access code, and that code would be valid for 15 minutes. We take the current time, add 900 seconds (15 * 60) and that's when the code expires. Here it also makes sense to use UTC.

Note however that this is only applicable when this time period is measured in minutes or maximum hours, but not days. If, for example, the order could be returned within 30 days, it is reasonable to store just the return deadline.

The third case is when **something will occur (in the future) at some local time in some specific location**. Every word is important here.

Consider the following example: there is a daily train between Brussels and London departing at 08:52 (Central European Time, CET). When does that journey actually begin? This depends on the time zone in Belgium, particularly on daylight savings time in that time zone.

Let's look at the four days around the time when DST changes (early in the morning of March 31st, 2024, between the second and third rows).

Central European Time	*Time in UTC*
2024-03-29 08:52:00	2024-03-29 07:52:00
2024-03-30 08:52:00	2024-03-30 07:52:00
2024-03-31 08:52:00	2024-03-31 06:52:00
2024-04-01 08:52:00	2024-04-01 06:52:00

Suppose that we want to build a ticket booking system. We need to add a record about the train that is scheduled to run on those days. What timezone should we use?

The answer here may be counterintuitive: we may want to actually store the local scheduled departure time (that does not change). Why? If we know when the daylight savings time changes, can't we just convert it to local time on the fly?

The problem is that DST is not stable over the years. Many countries have historically changed their approach to DST by moving the dates, or abandoning DST completely. Even for Central European Time, there are ongoing negotiations to abandon DST, and things like that sometimes happen very quickly. If you were to use UTC here, you'd have to do a careful adjustment of the exact time, which is error-prone.

Again, note that we're talking about scheduled trains here! Another piece of information we may want to store is when the train has actually departed. This one would be recorded in UTC, because it is a "when something actually happened" case, as described above.

To recap:

- use UTC to record things that actually happened (with precision of up to a second or even a fraction of the second);
- use UTC for things that will happen N seconds, minutes, or hours from now;
- use local time for things that will happen at a certain local time, no matter what the DST situation is: appointments, train rides, conference talks, etc. In that case you also need to record the corresponding time zone.

Here is an example of first two cases:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Order	When exactly was the Order **placed**?		
date/time (in UTC)	*"2024-11-26 19:44:00"*	⟨TBD⟩	
SMSCode	When does the SMSCode **expire**?		
date/time (in UTC)	*"2024-11-27 19:44:00"*	⟨TBD⟩	

The third case is more complicated. Imagine that our system sells tickets to concerts around the world. We want to store and display the door time. This is a local time in a specific city. Here is the list of **anchors**:

Anchor	*ID example*	*Physical table name*
City	1, 2, 3, ...	⟨TBD⟩
Timezone	*"Europe/ Brussels"*	Stored in tzdata (time zone database)
Concert	1, 2, 3, ...	⟨TBD⟩

Note that the "Timezone" anchor is special: it is stored in the system time zone database. See "Well-known anchors" chapter below.

List of **attributes**:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
City	What is the **name** of the City?		
string	*"Antwerp"*	⟨TBD⟩	
Concert	When do the **doors open**? (in the city time zone)		
date/time	*"2024-10-31 18:30:00"*	⟨TBD⟩	

And a list of links:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Timezone : City	**1:N**	A City can have *only one* Timezone A Timezone can cover *several* Cities	⟨TBD⟩
City : Concert	**1:N**	A Concert can take place in *only one* City A City can host *several* Concerts	⟨TBD⟩

For the physical schema for those examples, see "Handling time in the physical model" below.

3

"HELLO WORLD" USE CASE: PODCAST CATALOG

Let's design our first toy but realistic use case: a catalog of podcasts.

We're going to design an MVP: a minimum viable product. More complex use cases would be found later in the book.

We discuss how to extend this scope with more interesting features (e.g., categories) later in this section.

We're only going to build a logical model here. In the second part of the book we'll cover the physical design directly based on this model.

3.1 BUSINESS REQUIREMENTS

Let's use a screenshot of an episode from Apple Podcasts and see what information we have here (see the next page).

Here is a list of elements that we can see here:

- cover image;
- air date (*16 January 2025*);
- episode number (26);
- length (1 hr 3 min);
- episode title (*"Debanking explained"*);
- name of the show (*"Complex Systems with Patrick McKenzie"*);
- episode description (*"In this episode, . . . "*).

If you scroll down, there will be a bit more information, but let's begin with this. Oh wait, the actual audio file should prob-

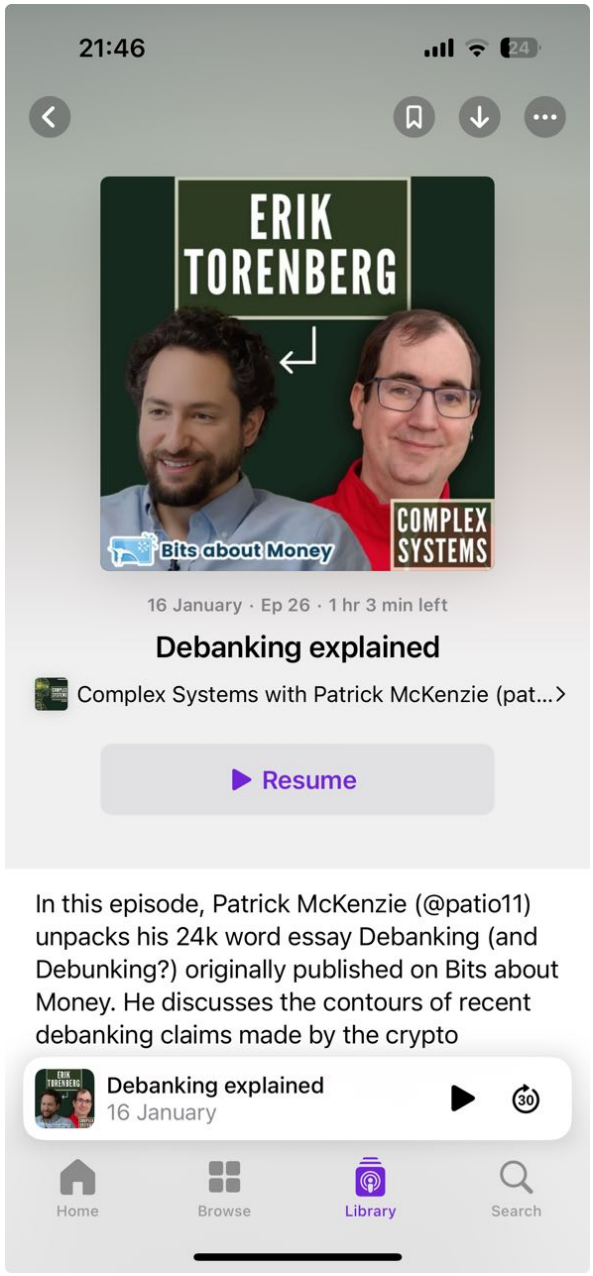


Figure 3.1: An example of a podcast screenshot

ably also be stored someplace. Good that we've thought about that.

A basic workflow for new podcast owners would be:

- sign up, login;
- add a new show;
- upload the first episode of the show;
- upload more episodes later on;

We're not going to specify the requirements too strictly: maybe we'll even find out later on that we've missed something. Building a logical model is itself a part of the process of clarifying requirements.

3.2 ANCHORS

The next step in the process is to find some anchors.

We need to find things that we could add and count. One suggestion is to imagine how we're going to brag about our business. *"Our system has 20,000 users!" "There are 2,000 podcast episodes in total!" "We have 500 shows in the catalog!"*

Here they are, the first three anchors:

Anchor	*ID example*	*Physical table name*
User	1, 2, 3, ...	⟨TBD⟩
Show	1, 2, 3, ...	⟨TBD⟩
Episode	1, 2, 3, ...	⟨TBD⟩

3.3 ATTRIBUTES

Next step in the process is to find the attributes corresponding to our anchors. The question that you should be asking is, "where is it stored?" We re-read the business requirements, repeatedly finding a way to ask this question. Every time this question makes sense, we know there is an attribute.

For example:

> "Here is a list of elements that we can see here:
>
> - cover image; (*where is it stored?*)
> - air date ("16 January"); (*where is it stored?*)
> - episode number (26); (*where is it stored?*)"
> - *and so on*.

A *User* is an ubiquitous anchor. It's hard to find a system that does not have some sort of users. Sometimes it's even different kinds of users. We do not talk about users in our draft of business requirements, but before we actually implement the system, we should. A lot of information about the users could be collected, but let's cut it down ruthlessly, leaving only the two essential attributes:

- their email address, also used as a login name;
- and their password.

Upon re-reading the previous section and asking "*where is it stored?*", we find the following list of potential attributes for episodes:

- a cover image;
- an air date;
- an episode number;
- run length;
- a title;
- a description;
- an audio file;

Also, in the screenshot we see only one attribute of show:

- its name.

It is clear that a lot of information is missing here: adding it would be a useful exercise for the interested reader.

To confirm that each attribute is in fact an attribute, let's write them down in the format described above. We'll use human-readable questions and example values to help with documenting.

Read through this table, paying attention to the questions, data types and example values. Do they make sense?

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
User	What is the **email address** of the User?		
string	*"somebody@example.com"*	⟨TBD⟩	
User	What is the encrypted **password** of the User?		
string	*"$2a12R9h/cIPzogi."*	⟨TBD⟩	
Show	What is the **name** of the Show?		
string	*"Complex Systems with Patrick McKenzie"*	⟨TBD⟩	
Episode	What is the **cover image** of this Episode?		
binary blob	*[JPEG file contents]*	⟨TBD⟩	
Episode	What is the **air date** of this Episode?		
date	*2025-01-16*		
Episode	When was the Episode uploaded? (in UTC timezone)		
date/time	*2025-02-14 00:25:52*		
Episode	What is the **runtime** of the Episode, in seconds?		
integer	*3780*		
Episode	What is the **sequence number** of the Episode?		
integer	*26*		
Episode	What is the **title** of the Episode?		
string	*"Debanking explained"*		

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Episode string	What is the **description** of the Episode? *"In this episode, we..."*		
Episode binary blob	What is the **audio file** of the Episode? *[MP3 file contents]*		

The questions make sense, and the example values are also plausible. There are two things to notice though.

First, you may be somewhat surprised with the audio file attribute and maybe also the cover image: should those things go into the database? We'll discuss this later when we'll talk about the physical design.

As well, we'll have to discuss the question of the "episode runtime" later, also in the physical design chapter. We'll find that this is a special sort of attribute.

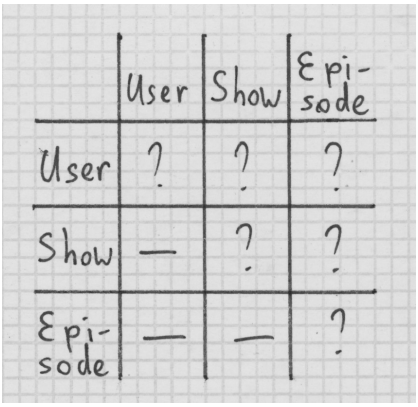
Note that we did not find any new anchors here, probably because the business domain is so simple. When you model a more complicated system, you'll routinely discover new anchors that you've missed on the first step.

3.4 LINKS

The next step is looking for links: connections between any two anchors.

To learn how to find the links, we can just look at all possible pairs of anchors and try to imagine if there is a link between the two. You also need to check for possible links between an anchor and itself.

For N anchors there are N * (N - 1) / 2 + N such candidates. For 3 anchors it's 6 pairs, for 4 anchors it's 10 pairs, and so on. A square format helps to think about that:



Here are the six possible pairs to think about (corresponding cells are marked with "?"):

- **User : User**;
- **Show : Show**;
- **Episode : Episode**;
- **User : Show**;
- **User : Episode**;
- **Show : Episode**;

Note that it's possible that there is more than one link between each two anchors.

Can users do something with users? In a social media system there would probably be some sort of friendship, subscription, or blocking feature, but in our MVP, no. Skip.

Can a Show be somehow connected to another Show? Not in a way I can think of at the moment. Skip.

Can an Episode be somehow connected to an Episode? Yes, but not in the MVP. Skip.

How could Users and Shows be connected? Well, users (podcasters) can register a new show before they start uploading. So, it seems like users own the shows, and shows are owned by users. Let's write this down in a formalized way.

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
User : Show	**1:N**	A User can own *several* Shows A Show can be owned by *only one* User	⟨TBD⟩

Because the sentences mention "several" and "only one" here, we know that the cardinality is 1:N. We write the 1-side anchor first, so it's "**User** : **Show**", and not vice-versa.

Is there a link between a User and Episode? It seems logical that users upload the episodes, so we want to record this fact. But there is also another possibility: you can just say that users can upload episodes only to the shows that they own, and it's not necessary to record this. But we can make a counterargument: what if later we introduce a team-based access control, so that we can have one user managing a show, and multiple users who can upload new episodes. Which is right?

The answer is that it's your call. It's going to be just as you decide. You can ask your stakeholders for confirmation: maybe it's an important feature. If you're the stakeholder, you can decide either way.

For the purposes of this chapter, let's decide to store the information about the user who uploaded the episode, even though it's (currently) always the same user. In this tiny system it doesn't really matter, but it will matter in more complicated systems. Here it goes:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
User : Episode	**1:N**	A User can upload *several* Episodes An Episode can be uploaded by *only one* User	⟨TBD⟩

Finally, what about the *Show / Episode* pair? There is a very clear link between them: a show includes several episodes. Let's write this down:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Show : Episode	**1:N**	A Show includes *several* Episodes An Episode can be included in *only one* Show	⟨TBD⟩

So, we've checked all possible combinations. In a more advanced system, one could easily imagine more links, for example, "*A User likes an Episode*" and "*A User subscribes to a Show*". But we're working on an MVP here.

3.5 CROSS-CHECKING THE REQUIREMENTS

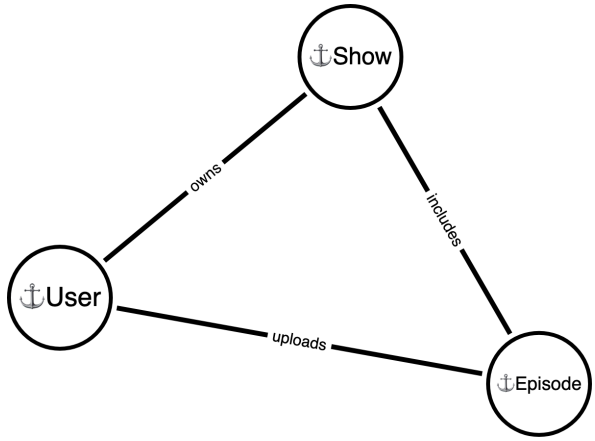
How can we make sure our database schema is correct? One validating step is to go back to the business requirements and highlight the fragments that we've got covered. Then we'll have a chance to notice any missing elements. Let's use bold for highlighting:

> "Here is a list of elements that we can see here:
>
> - **a cover image**;
> - **an air date** ("16 January");
> - **an episode number** (26);
> - **length** (1 hr 3 min);
> - **an episode title** ("Debanking explained");
> - **the name** of the **show** ("Complex Systems with Patrick McKenzie");
> - **an episode description** ("In this episode, . . . ").
>
> [. . .] the actual **audio file** should probably be stored somewhere. [. . .]
>
> A basic workflow for new podcast **owners** would be:
>
> - sign up, login;
> - **[owners] add** a new **show**;
> - **[owners] upload** the first **episode** of the show;
> - later, upload more episodes to the show;"

I do not see any elements missing from here.

3.6 A DIAGRAM

We don't use diagrams for modeling, but many people like visual representations. Here is a simple logical diagram, created in a tool called Arrows.app:



Circles correspond to anchors (with a little anchor emoji). Lines correspond to links, labeled by the link verb.

We do not show attributes here, because they don't add a lot of information.

Also, the connecting lines do not have arrows, because both anchors in a link are equal.

You can use your favorite diagramming tool to create the diagrams if you need. You can even use one of the standard notations for ERD diagrams, if this is required in your organizational context.

3.7 ARE WE THERE YET?

A fraction of this book's readers may not even be interested in physical schemas. Imagine that you actually want to hire somebody to write our system for you (or ask an LLM to do that). In that case it would be enough to write down the business requirements and to include the logical model. The logical model will unambiguously define the scope of the system you want to build.

Whoever would be implementing the system will implement the physical schema on their own. They will need to take into account technical requirements such as the estimated number of episodes and users; the exact type of the database server they're going to use; their preferred table design strategy and naming strategy, etc., etc.

If you're going to also work on the implementation, the second half of the book is for you. We'll discuss building a physical schema for this toy system later in the "Podcast catalog: a physical schema" chapter.

3.8 EVOLVING THE SYSTEM

We've discussed the MVP: it consists of 3 anchors, 10 attributes, and 3 links. Along the way we noted many feature ideas that come to mind. How do we evolve the system using this approach?

On a logical level, one can just add new elements to our catalog of anchors, links and attributes, exactly like we did in this chapter. Then we'll just have to decide how to migrate the physical schema. We discuss this in the "Evolving your database" chapter later in the book.

4

BUILDING A PHYSICAL SCHEMA

So, now that we have a catalog of anchors, attributes and links, we can build a physical schema: the actual database tables. When you have a catalog, 75% of work is done. Constructing database tables is pretty straightforward at that point.

4.1 MANY TABLE DESIGN STRATEGIES ARE POSSIBLE

Imagine that you want to build a relatively simple, but non-trivial application, for example, a to-do list. You can write down a set of functional requirements for the app and you can define a logical model (exactly as described in the previous chapters). You do all that without any actual software development first.

The next step is to design the physical table schema. But let's do a thought experiment: suppose that we hire a dozen teams of software developers, give them the requirements document, and ask them to implement it as they see fit.

They are free to choose their favorite database server and to design tables any way they like. The only restriction is that the requirements must be implemented fully, without changes, or additional features, and so on.

There would most probably be twelve different physical schemas. Some of them may look very similar to each other, but a couple would probably look really exotic.

Physical schemas would be different because different teams would use different **table design strategies**. A table design strategy is a set of rules that prescribes how to store anchors, how to store attributes, and how to store links.

Here are the most popular table design strategies:

- **Table-per-anchor**: one table per anchor, one table per M:N link. See below for a detailed description. We're going to use this as the basic strategy in the book;
- **Side tables**: a simple extension of the previous one.
- **Table-per-attribute** (or: physical anchor modeling): one table per attribute, one table per anchor, one table per link;
- **JSON-based schema**: several attributes of the same anchor could be stored together in a single column having a physical JSON type;
- **Entity-Attribute-Value (EAV)**: several attributes of the same type, belonging to the same anchor, could be stored together in a single table;

Each table design strategy is a pure approach. In the industry practice, pure approaches are almost never used exclusively. Instead, people would use almost every approach that they are familiar with, all in the same database. When the need arises, they choose one of the strategies, according to what the situation demands. This is a typical scenario that happens in long-lived systems over the years.

In this book, we're going to discuss the table-per-anchor strategy extensively. It is the easiest and most straightforward approach of all, so great for teaching and works well in practice, but we'll also cover other strategies and the driving forces behind them.

5

TABLE-PER-ANCHOR TABLE DESIGN STRATEGY

Table-per-anchor strategy is probably the **most straightforward way** of building tables:

- one table per anchor;
- one table per many-to-many links;
- each attribute is a column in its anchor's table;
- each only-one-to-many link is a column of a table for the "many" anchor.

This approach corresponds to the third and fourth normal forms as defined in the relational theory. It is roughly what they would teach you in the beginner level university courses on database design.

The resulting tables are clean, understandable, logical and error-proof. From the physical point of view, this design is neutral, it is not optimized for any specific access pattern, except for understandability.

5.1 ACTION PLAN

Creating tables from the logical model is straightforward. Here are the next actions:

Step 1. Insert table names in the list of anchors;
Step 2. For each attribute, add the column name and choose the data type;
Step 3. For each many-to-many link, choose the name of the table;
Step 4. For each only-one-to-many link, choose the name of the column;

Step 5. Write down the final SQL `CREATE TABLE` statements, using choices made above;
Step 6. you're done.

Each step is described in its own section.

We're going to use the same podcasting catalog system for illustration.

5.2 ANCHORS: CHOOSING TABLE NAMES

We need to go over our anchors and choose the table names for each of them. Here is the list of anchors:

Anchor	*ID example*	*Physical table name*
User	1, 2, 3, ...	`users`
Show	1, 2, 3, ...	`shows`
Episode	1, 2, 3, ...	`episodes`

Choosing the names for the anchor tables is usually super simple: just use the plural noun.

Now we're going to write down the partial anchor table definition. Attribute columns will be added later.

We use the same type of IDs (integer numbers), so all three tables would start out looking the same.

```
CREATE TABLE episodes (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY
);

CREATE TABLE shows (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY
);

CREATE TABLE users (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY
);
```

5.3 ATTRIBUTES: CHOOSING COLUMN NAMES

We repeat the list of attributes here, with the first four columns unchanged. In this section we'll discuss the "Physical column" column.

"Physical type" would be discussed in the next section.

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
User	What is the User's **email address**?		
string	*"somebody@example.com"*	`users.email` `VARCHAR(64) NOT NULL`	
User	What is the User's encrypted **password**?		
string	*"$2a12R9h/cIPzogi."*	`users.password` `VARCHAR(128)`	
Show	What is the **name** of the Show?		
string	*"Complex Systems with Patrick McKenzie"*	`shows.name` `VARCHAR(192)`	
Episode	What is this Episode's **cover image**?		
binary blob	*[JPEG file contents]*	⟨TBD⟩	
Episode	What is this Episode's **air date**?		

date	*2025-01-16*	`episodes.`
		`air_date`
		`DATE NULL`	

Episode	When was the Episode uploaded? (in UTC timezone)		

date/time	*2025-02-14 00:25:52*	`episodes.`
		`uploaded_at`
		`TIMESTAMP(6) NOT NULL`
		`DEFAULT CURRENT_TIMESTAMP(6)`	

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Episode	What is the Episode's **runtime** in seconds?		
integer	*3780*	`episodes.runtime` `INTEGER`	
Episode	What is the Episode's **sequence number**?		
integer	*26*	`episodes.seq_number` `INTEGER`	
Episode	What is the Episode's **title**?		
string	*"Debanking explained"*	`episodes.title` `VARCHAR(192)`	
Episode	What is the Episode's **description**?		
string	*"In this episode, we..."*	`episodes.description` `TEXT`	
Episode	What is the Episode's **audio file**?		
binary blob	*[MP3 file contents]*	⟨TBD⟩	

Column names should be short, because they would be used in SQL queries, and they are generally typed by hand. For example, "`episodes.description''` is a great column name. ("`episodes`" part is the same table name that we've chosen in the list of anchors).

Some column names are "forbidden", because they are reserved as SQL keywords. One example is the word "`date`". It is reserved, so we had to use "`episodes.air_date`" instead. Thankfully, it makes this name a bit more understandable.

For attributes of "yes/no" type, you should use names that look like questions:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
TodoItem	Is this To-do Item marked as done?		
yes/no	*yes*	`todo_items.is_done`	`INTEGER NOT NULL DEFAULT 0`

For date/time attributes it's natural to use a verb-based name, such as "`episodes.uploaded_at`".

Unfortunately, sometimes you have to choose a name that may not be immediately understandable. This is also why we suggest starting with the logical model. There is just more space for explanation in the human-readable document than in a short_machine_readable_name.

Most of the time the names need to be concise, but sometimes you really want to distinguish between two similar attributes. One common example is price: you may want to specify if the tax is included: "`line_items.price_after_tax`" and "`line_items.price_before_tax`". This would be documented in the question "*What is the price of this Item (before tax)?*", but this is such an important question that it's worth mentioning in the column name directly.

5.4 ATTRIBUTES: CHOOSE COLUMN DATA TYPES

In the "Data types and types of data" section we discussed the logical data types of attributes. Physical data types are similar, but not quite.

This book covers MySQL 8.0 and PostgreSQL 15. You will most probably choose one of those two for your project. In a separate chapter I'll give some advice on what to expect from other systems, such as Oracle or SQLite, including non-relational ones such as MongoDB or Cassandra, and even some more involved, such as Amazon DynamoDB.

This chapter is the first where it makes sense to talk about specific database servers. You may notice that we went through the entire "Building a logical model" chapter without ever mentioning a specific database server. This is by design! Having a logical model of your database is a necessary prerequisite for any database engineering.

Why do we need to choose a physical data type? Because `CREATE TABLE` SQL statement requires specifying the data type for each column.

So, let's go over all the logical data types and talk about the corresponding physical data types. First, let's jump ahead a bit with this summary table. We'll discuss it in more detail in the following few sections.

On the next page you can see a summary of most commonly used physical types that should work in many databases. However, you should read documentation for your favorite database to learn what data types it supports and encourages.

5.4.1 *Recommended physical types for SQL databases: summary*

Strings: `VARCHAR(N) NOT NULL DEFAULT ""`
(choose *N*: max. string length).

Integer numbers: `INTEGER NULL`

Monetary amounts:

- `DECIMAL(15, 2) NULL`
  (if you only care about USD or EUR, up to 10 trillion.)
- `DECIMAL(17, 4) NULL`
  (to support all the world's traditional currencies.)
- `DECIMAL(M, D) NULL`
  (choose M as the maximum number of digits including minor units, and D as the number of digits in the minor unit).

Numeric values: `DECIMAL(M, D) NULL`
(choose M, the maximum number of digits, incl. the fractional part, and D, the number of digits in the fractional part)

Yes/no values: `INTEGER NOT NULL DEFAULT 0`

Either/or/or values:
`VARCHAR(24) NOT NULL DEFAULT "<initial-value>"`
(choose *'<initial-value>'*).

Dates: `DATE NULL`

Date with time: `DATETIME NULL`
(both UTC and local time zone)

Timestamps (NOT an attribute value, see below):
`TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)`

Binary blobs: `BLOB NULL`
(NB: Binary blobs that are image files and other media may be

stored in some cloud object storage, such as Amazon S3. In a smaller application, though, it may still make sense to keep binary blobs directly in the database.)

5.4.2 *Strings*

Your best choice is `VARCHAR(N) NOT NULL DEFAULT ""`. The main question is how to choose the N: the maximum number of characters that attribute values will need. Do not worry, you can change that later if you encounter longer values in practice. If the attribute value exceeds N characters, your server will signal an error when you try to insert or update the attribute value.

For the title of the book we chose "`VARCHAR(256)`", a good round number. For the name of the author, "`VARCHAR(128)`"; another round number that is a little bit smaller. It's fine to choose N with some extra slack.

However, don't go overboard. If you choose "`VARCHAR(100000)`" for the book title, two things would happen:

- First, people would be confused. What do you mean by this? That sounds more like the entire text of the book, rather than a title.
- Second, think about what happens when somebody actually submits a value this long. Will it break some part of your website, maybe visually, or even break some other component that does not anticipate such a long value?

For each data type we're going to discuss how to deal with the attribute values that are not set. They may be unknown, not provided, discarded, missing or absent. We'll discuss this topic (missing data, NULLs, sentinel values, etc) in a separate section, but we need to make a decision for strings.

The recommended decision is that the physical column is going to be "not-NULL", and if a specific value is not provided, we use an empty string. This is what the "`NOT NULL DEFAULT ""`" part means.

5.4.3 *Integer numbers*

Your best choice is `INTEGER NULL`. It may contain an integer number in the range roughly of plus-minus two billion.

Here we decide that if the value is not set (unknown, missing, not provided, etc.), then we will have a special NULL value in the physical column.

5.4.4 *Monetary amounts*

For monetary amounts, your best choice is `DECIMAL(M, D) NULL`. The main question is how to choose the M and D. M is the maximum total number of digits that this value can store. D is the number of digits after the decimal point.

For example, if you choose the type `DECIMAL(5, 2)` then you will be able to store values such as "123.45" (total of five digits, two digits after the decimal point).

You won't be able to store "*2345.60*" (too many digits in total), for instance, or "*345.672*" (too many digits after the decimal point).

So, how to choose the M and D? Most currencies in the world have two digits after the decimal point. If you only need to support single-currency amounts, choose $D = 2$, or whatever your currency needs. One trillion dollars needs 13 digits. Thus, D = 13 + 2 = 15. This will be enough to keep accounting for the mega-unicorn companies.

In short: choose `DECIMAL(15, 2) NULL` if you only care about USD or EUR. Choose `DECIMAL(17, 4)` if you need to support all the world's traditional currencies.

NB: Do not use `FLOAT`.

5.4.5 *Numeric values*

Your best bet is `DECIMAL(M, D) NULL`. For D, choose the maximum fractional precision you need. For M, find the largest number that you can reasonably expect, add D, and you get your M.

For example, if you store temperature with up to one digit precision, choose `DECIMAL(4, 1)`. This will cover all human-

compatible temperatures: +999 maximum, one digit after the decimal point.

5.4.6 *Yes/no values*

Your best bet is `INTEGER NOT NULL DEFAULT 0`. Use zero for "no", one for "yes". Do not use any other values.

5.4.7 *Either/or/or values*

Reminder: here we're talking about some sort of statuses or states. For example, you can have an *Order* anchor that could be in the following states:

- *Initiated;*
- *Paid;*
- *Fulfilled;*
- *Sent;*
- *Received;*
- *Cancelled.*

First, write down the list of all values of this attribute that you currently know. There should be half a dozen and up to a dozen states.

A rule of thumb: when you add a new possible value to the either/or attribute, you must change the code of your application. If you don't need to change the code then it's not a good either/or/or value.

Your best bet for the physical data type is:

```
VARCHAR(24) NOT NULL DEFAULT "<initial-value>"
```

We choose 24 because it's long enough for most reasonable status values. Of course, the number is completely arbitrary: you can choose 16, but then you may be too cryptic, or 32, but then you would be too verbose. Start with 24 and see how it goes.

The '<*initial_value*>' should be the initial status of the thing that this belongs to. For example, newly-created *Orders* would have the status of *"initiated"*. Use this, then:

```
VARCHAR(24) NOT NULL DEFAULT "initiated"
```

5.4.8 *Dates*

Your best bet is `DATE NULL`. If the date value is not provided, a `NULL` would be used instead.

5.4.9 *Date with time in UTC timezone*

See the "Handling time in the logical model" chapter for discussion. Reminder: this is used to handle two cases:

- when something actually happened;
- when something will happen N seconds from now;

Your best bet is `DATETIME NULL`. Note that before inserting or updating you must convert the time to UTC in your application.

5.4.10 *Date with time in a specific timezone*

See the "Handling time in the logical model" chapter for discussion. Reminder: this is used to handle the third case:

- when something will happen at a local time **at a specific location**.

Your best bet is also `DATETIME NULL`.

5.4.11 *Timezone names*

You also need to decide how you will be storing the associated timezone information. The most straightforward way is to just store the string in a standard notation such as *"Europe/Lisbon"*. Then in your application you would use the local time from the respective column and the timezone information to convert into another timezone, for example into UTC.

You have to think carefully here: your database server also has its own time zone, most probably the one where it's located. For example, your database server is hosted in Berlin. But your business may be international: for example, you want to store information about concert times in different cities of the world. So, your database server's own timezone is irrelevant, and that's

why you have to store the time zone identifier for each attribute of type "date with local time".

5.4.12 *Binary blobs*

MySQL 8: Your best best is `MEDIUMBLOB NULL`. (Up to 16 Mb.)
PostgreSQL 15: Your best bet is `BYTEA NULL`. (Up to 1Gb.)

Blobs could be used to store all kinds of file data, such as JPEG images or PDF files. For example, if you generate PDF invoices you may want to store them for future reference.

However, **storing files in the database is generally not a good idea**. If you expect that you will have more than a few thousand files, be prepared to switch to cloud storage such as Amazon S3, or just write the files to the file system. This topic is outside of the scope of this book.

5.5 LINKS

As you may remember from the "Cardinality" chapter, links come in a variety of types:

- Many-to-many links (M:N), such as "A *Developer is assigned to a Project*";
- One-to-many links (1:N), such as "A *User placed an Order*";
- One-to-one link (1:1), such as "*A Person is married to a Person*".

We store them in two different ways.

5.5.1 *One-to-many (1:M) links*

One-to-many links, such as "A *User placed an Order*", are stored in the same table as the anchor from the "many" side of the link.

In this example:

- *A User can place several Orders*;
- *An Order can be placed by only one User*.

So, the *User* is on the "only one" side, while the "*Order*" is on the "many" side, so we will add the following SQL snippet to the "orders" table definition (see in bold):

```
CREATE TABLE orders (
  id INTEGER NOT NULL PRIMARY KEY,
  user_id INTEGER NOT NULL,
  INDEX (user_id),
  -- other attributes and links go here
);
```

In the list of the links we're going to write down the short notation: "`orders.user_id`".

Note that there could be multiple 1:N links between the same two anchors. For example:

- *An Expense is requested by an Employee;*
- *An Expense is approved by an Employee;*
- *An Expense is paid out by an Employee;*

In this case we would need to find three column names that are easy to distinguish, for example:

```
CREATE TABLE expenses (
  id INTEGER NOT NULL PRIMARY KEY,
  requested_by INTEGER NOT NULL,
  approved_by INTEGER NULL,
  paid_out_by INTEGER NULL,
  -- other attributes and links go here
);
```

Choosing short and readable column names can be somewhat challenging in such situations, but this is something that you have to learn to do.

5.5.2 *An ID cannot be an attribute value*

Back in the "What is not an attribute?" section, I said that attributes cannot contain anchor IDs.

According to "`orders`" table definition above, the "`user_id`" column looks very much like an attribute column. When you speak informally, you're going to use expressions like, "the *customer that placed this order?*", or "the *employee's manager*", or "the *Post's author*", etc.

Those sound very much like, "*what is the name of the item?*", "*an Employee's name*" or "the *text of the Post*" that you use when you talk about attributes.

It's important not to confuse the two. An ID that relates to another ID is a link.

5.5.3 *Many-to-many (M:N) links*

Many-to-many links, such as "A *Project is assigned to a Developer*", are stored each in their own table.

Here is how this table will look like:

```
CREATE TABLE project_developers (
  project_id INTEGER NOT NULL,
  developer_id INTEGER NOT NULL,
  PRIMARY KEY (project_id, developer_id),
  INDEX (developer_id)
);
```

As discussed in the "Links: definition" chapter, links are symmetrical by nature. That means that we might as well design this table the other way around:

```
CREATE TABLE developer_projects (
  developer_id INTEGER NOT NULL,
  project_id INTEGER NOT NULL,
  PRIMARY KEY (developer_id, project_id),
  INDEX (project_id)
);
```

The difference is highlighted in bold.

Both definitions are correct. You must decide which one you like more and just use it. People are used to this and nobody will complain. One of the possible conventions is to begin with the anchor that is alphabetically first (in this case it would be "`developer_projects`").

Naming link tables has one more challenge. Both proposed names, "`developer_projects`" and "`project_developers`", are missing one of the most important parts of a link: the verb.

The problem here is that if you tried to add the verb, the name would easily become too long.

```
CREATE TABLE developers_assigned_to_projects (
  ...
);

CREATE TABLE projects_assigned_to_developers (
  ...
);
```

Those names are very descriptive, but unfortunately, they are quite long. People will have to type those names as they write database queries, and they will complain.

So, omitting a verb is a common practice, but that works only if there is only one link between two anchors. But what should you do if there are several?

Here is a common example of of three (M:N) links between the same two anchors:

- *"A User likes Posts"*;
- *"A User bookmarks Posts"*;
- *"A User hides Posts"*.

Those three links need three different tables. How do we name them? We can spell it out:

- `users_like_posts`;
- `users_bookmark_posts`;
- `users_hide_posts`;

Those names are on the verge of being too long. A common practice is to omit the name of one anchor, and to convert the verb into a noun:

- `user_likes`;
- `user_bookmarks`;
- `user_hides`;

Again, because the links are symmetrical, it's not clear which anchor to omit:

- `liked_posts`;
- `bookmarked_posts`;
- `hidden_posts`.

You can even omit both anchors, if the meaning of the table name stays clear:

- `likes`;
- `bookmarks`;
- `hides`.

Choosing short and readable names can be somewhat challenging, but this is something that you have to learn to do.

By the way, it's common to sometimes choose an unfortunate table name from the very beginning. That name has every chance to remain part of the database forever, because people very rarely decide to migrate a table only to rename it. Unfortunately, commonly used databases rarely support aliasing table names to facilitate such migrations.

5.6 PODCAST CATALOG: A COMPLETE PHYSICAL SCHEMA

To finalize our toy problem, let's build the tables for our podcast catalog. We'll also discuss how to store the audio files and cover images.

Here is a copy of the list of anchors from "Anchors: choosing table names", for reference.

Anchor	*ID example*	*Physical table name*
User	1, 2, 3, ...	`users`
Show	1, 2, 3, ...	`shows`
Episode	1, 2, 3, ...	`episodes`

And here is a copy of the list of attributes (unchanged, for reference):

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
User	What is the User's **email address**?		
string	*"somebody@example.com"*	`users.email` `VARCHAR(64) NOT NULL`	
User	What is the User's encrypted **password**?		
string	*"$2a12R9h/cIPzogi."*	`users.password` `VARCHAR(128)`	
Show	What is the **name** of the Show?		
string	*"Complex Systems with Patrick McKenzie"*	`shows.name` `VARCHAR(192)`	
Episode	What is this Episode's **cover image**?		
binary blob	*[JPEG file contents]*	⟨TBD⟩	
Episode	What is this Episode's **air date**?		

date	*2025-01-16*	`episodes.`
		`air_date`
		`DATE NULL`	

Episode	When was the Episode uploaded? (in UTC timezone)		

date/time	*2025-02-14 00:25:52*	`episodes.`
		`uploaded_at`
		`TIMESTAMP(6) NOT NULL`
		`DEFAULT CURRENT_TIMESTAMP(6)`	

Episode	What is the Episode's **runtime** in seconds?		

integer	*3780*	`episodes.`
		`runtime`
		`INTEGER`	

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Episode	What is the Episode's **sequence number**?		
integer	*26*	`episodes.seq_number` `INTEGER`	
Episode	What is the Episode's **title**?		
string	*"Debanking explained"*	`episodes.title` `VARCHAR(192)`	
Episode	What is the Episode's **description**?		
string	*"In this episode, we…"*	`episodes.description` `TEXT`	
Episode	What is the Episode's **audio file**?		
binary blob	*[MP3 file contents]*	⟨TBD⟩	

And here is a list of links with information about physical storage, according to the rules from the "Links" section:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
User : Show	**1:N**	A User can own *several* Shows A Show can be owned by *only one* User	`shows.owner_user_id`

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
User : Episode	**1:N**	A User can upload *several* Episodes An Episode can be uploaded by *only one* User	`episodes.uploader_user_id`
Show : Episode	**1:N**	A Show includes *several* Episodes An Episode can be included in *only one* Show	`episodes.show_id`

5.6.1 *CREATE TABLE statements*

Here are the SQL `CREATE TABLE` statements, assembled from all the pieces presented in the lists:

```
CREATE TABLE users (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,
  email VARCHAR(64) NOT NULL,
  UNIQUE (email),
  password VARCHAR(128) NOT NULL
);

CREATE TABLE shows (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(192) NOT NULL,
  owner_user_id INTEGER
);

CREATE TABLE episodes (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,
  air_date DATE NULL,
  uploaded_at TIMESTAMP(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
```

```
  runtime INTEGER,
  seq_number INTEGER,
  title VARCHAR(192),
  description TEXT,
  uploader_user_id INTEGER,
  show_id INTEGER
);
```

Bolded are the names of the columns and tables that correspond to the anchors, attributes, and links.

5.6.2 *Audio file and cover images*

On the logical level, an audio file of an episode, and the cover image of a show are attributes of a binary blob type (basically a file contents).

On the physical level, such attributes can be stored outside of the main relational database. It is common to store such files, for example, in a cloud-based object storage system such as Amazon S3. Handling files in S3-like systems is out of scope for this book.

Technically, it is possible to store even large blobs in a database (using the BLOB physical type), but this is not generally recommended.

Here is how you would describe such attributes:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Episode	What is this Episode's **cover image**?		
binary blob	*[JPEG file contents]*	`s3://podcasts/cover-images/${episode_id}.jpg` *S3 object*	
Episode	What is this Episode's **audio file**?		
binary blob	*[MP3 file contents]*	`s3://podcasts/audio-files/${episode_id}.mp3` *S3 object*	

The table heading ("Column name") is not general enough, but this should not be too confusing.

We use an S3 URL template, which is human-readable. If more information is needed, we can just provide a link to the detailed documentation.

5.7 PHYSICAL ID DESIGN

5.7.1 *Case study: a tiny CMS*

Let's collect the entire logical model of our tiny CMS with several slugs per page and discuss its physical design. First, we'll choose names for the physical tables and columns and specify physical types according to the "Attributes: choose column data types" section above.

Anchor	*ID example*	*Physical table name*
Page	1, 2, 3, ...	`pages`
LegacyURL	1, 2, 3, ...	`legacy_urls`

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Page	What is the **text** of this Page?		
string	*"Our company was established in 1983..."*	`pages.page_text` `TEXT`	
Page	What is the **URL slug** of this Page?		

string, unique	*"/about"*	`pages.url_slug`
		`VARCHAR(128)`
		`UNIQUE(url_slug)`	

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
LegacyURL	What is the **slug** of this LegacyURL?		
string, unique	*"/our-company"*	`legacy_urls.slug VARCHAR(128) NOT NULL UNIQUE (slug)`	

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Page : LegacyURL	**1:N**	A Legacy URL belongs to *only one* Page A Page can have *several* Legacy URLs	`legacy_urls.page_id`

The two new fragments of SQL that we used are "`UNIQUE(slug)`" and "`UNIQUE(url_slug)`" in the last two attributes.

```
CREATE TABLE pages (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  page_text TEXT NOT NULL,
  url_slug VARCHAR(128) NULL,
  UNIQUE (url_slug)
);

CREATE TABLE legacy_urls (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  slug VARCHAR(128) NOT NULL,
  page_id INTEGER NOT NULL,
  UNIQUE (slug)
);
```

What we did here is we declared a unique index on both the "`url_slug`" and "`slug`" columns. This index makes sure that the

URLs are unique. If you attempt to insert another row with an URL slug that already exists, you will get an error.

Additionally, searching by the value of this attribute is super fast.

In the "More on IDs and unique attributes" chapter earlier in the book, we talked about logical models of some use-cases:

- CMS with one URL per page and multiple URLs per page;
- Order numbers, and some concerns about industrial espionage.

As we discussed the table-per-anchor table design strategy in this chapter, we used the simplest approach to encoding ID: we just always used the "`INTEGER`" physical type. Let's now consider some variations that you must be aware of. Those variations only become tangible on the physical level.

5.7.2 *The maximum number of items*

An "`INTEGER`" physical type uses 4 bytes of storage (32 bits). It means that it can fit positive numbers up to 2.147 billion. That means that you cannot have more than 2.147 billion of some things (orders, order items, users, posts, etc.). This type also supports negative numbers down to the same limit, but they are usually not used.

If you change to "`UNSIGNED INTEGER`", you can have numbers up to 4.294 billion, so twice as many.

In many real-world databases this eventually becomes insufficient. Even the Earth's population is bigger than that number.

The next practical physical data type is called "`BIGINT`". It uses 8 bytes of storage (64 bits). This is more than the number of atoms that the Earth has, so it's practically infinite.

5.7.3 *Reaching the maximum number of items*

What happens when the anchor ID reaches the limit? Well, the corresponding part of your business breaks until you fix it. Namely, it's not possible to create new things. If you have an Order with *ID=2147483647*, you just won't be able to create an Order with the very next value, *ID=2147483648*. The application

will receive an error message from the database server and will return an error to the customer.

This is a surprisingly common reason for outages in real-world databases. Sometimes people set up some sort of monitoring for the current maximum value of IDs in all tables, but this monitoring can break or miss something. When this happens, you usually have to work against the clock to restore business operations. And the problem is that if your database has more than 2 billion of items, that usually means that your business is big enough for outages to have substantial consequences.

To fix that, you need to change the physical type of anchor ID, both in the main table, but also in the other tables that refer to that ID. So, for every link that refers to this anchor, you must also change the definition of the table that contains that link.

We'll talk about such database migrations in the next chapter, "Evolving your database". We'll also talk about table rewrites and why they usually require substantial time. Unfortunately, changing the data type of the column most often leads to the table rewrite — it's expensive and may cause a service downtime.

In other words, try to avoid finding yourself in a situation where you need to change the physical definition of anchor ID in a rush. To reduce this risk, you can a) monitor the maximum value of table columns that contain IDs and b) proactively change data types in tables that are getting closer to the limit.

5.7.4 *Space taken by IDs*

Why can't we just always use 64-bit integers for every anchor ID? Technically we can, but there are a couple of additional concerns: storage size and, maybe more importantly, density.

Anchor IDs are used not only in the main table, but also in other tables that refer to that anchor, mainly link tables. It means that we need to use the same data type for all columns that contain this anchor ID.

The first issue is disk space. Suppose that our system deals with restaurant orders. There are certainly fewer than 2 billion restaurants in the world, so we may want to use "`INTEGER`" data type (4 bytes). But we will have to specify the restaurant ID in the table that stores restaurant orders (because there is a link

"Restaurant can handle more than one Order; an Order can be placed in only one Restaurant").

Suppose that we have 1 billion orders in our database. It means that we need roughly 4 Gb of disk space to store the restaurant IDs of those orders. Were we to have used a "`BIGINT`" data type (8 bytes), we'd need 8 Gb of disk space: twice as much.

Those numbers slowly but surely add up. At some point you may find yourself in a situation where you would have preferred to save a bit of space in your storage.

Of course, nobody cares if you had 100 Gb and then saved some space to shrink storage to 80 Gb: that's not the point. The point is that with 100 Gb of space you could store, say, a billion orders; but now you can store 1.2 billion orders using the same space. This is a much more interesting way of looking at this: it means that you can delay spending money on storage upgrade.

5.7.5 *Disk space is time*

There is another reason why caring about data types may be important. Some database operations require reading the contents of the entire table. Such operations are relatively rare but important: for example, backing up table data and rewriting the table during migrations.

If reading 4 Gb of data from the disk takes X seconds, then reading 8 Gb of data takes twice as long. Again, we are not interested in just saving some time. We are interested in operations that cross some thresholds: for example, whether your database backup takes more than a workday versus a few hours (or vice versa). Such jumps can mean big process improvements, or big process bottlenecks.

Note that this is soft advice. It does not mean that you need to ration your bytes. Just keep in mind that following this advice may be a way out of undesirable situations, or of not getting yourself into such situations.

5.7.6 *Storage density*

Yet another practical reason to care about data type is storage density. Roughly speaking, table data is stored in a column-

by-column order. Let's see what happens when your database engine queries that data.

Due to the way computer memory works, keeping the data compact is one of the most effective ways to increase data processing performance.

Suppose that we have a standard link table that contains two IDs: for example, a project ID and a developer ID. If both IDs are 32-bit, one row takes roughly 8 bytes. Four kilobytes of memory would thus store around 512 records.

If both IDs are twice as large, four kilobytes of memory would store only 256 records. So, the memory is used more efficiently if data is stored compactly.

This is also a soft concern. Your database server and your hardware will go out of their way to handle whatever data you throw at them. But you may sometimes find that taking data size into consideration could bring some performance benefits.

5.7.7 *UUIDs as anchor IDs*

UUIDs is basically a huge randomly-generated number that looks like this: *"1ba690db-436d-42f8-b6e7-865617a8a8fc"*. You may have heard about using UUIDs in the context of databases, especially distributed databases.

You need to understand the reasons why UUIDs may be used in some applications.

An UUID is a huge 128-bit number. So if you generate a UUID using a good random number generator, you will get a number that will literally never be generated by anybody else in the world. An UUID is globally unique.

Imagine a heavily loaded system that creates new rows 100k times per second, or maybe even 10 times as fast. You need 100k new IDs every second, and those IDs must be unique. If we use the usual auto-incremented integer IDs, the database will have to carefully keep track of the current value of this ID. When a new record is requested, it's going to read the current value, increment it, and write the value back. That all happens very quickly, but if the insertion frequency is high enough, this may become a bottleneck.

A bottleneck is due to central coordination: there is exactly one place that holds the authority on allocating the IDs.

UUIDs do not need this central authority: basically each client can allocate unique IDs themselves and just tell those to the database server during insertion.

As a consequence, UUIDs allow spreading the insertion load across multiple servers. With lots of orders placed simultaneously, we can just use two servers that would each keep a copy of their orders. This is because UUIDs do not collide.

Reminder: this is something that is relevant to heavily loaded systems and to some distributed solutions.

On the flip side, UUIDs are huge: 128 bits, that is, 16 bytes each. As you remember, normal 32-bit integers take 4 bytes, and 64-bit integers take 8 bytes. So, you need additional space to store UUIDs. In the previous section, "Storage density", we discussed the effect this may have on performance.

As for the physical storage type, some database servers have direct support for a UUID column type. In other systems you can use something like `BINARY(16)` (a type that holds 16 bytes).

5.7.8 *Countries, currencies, languages: well-known anchors*

There are several **well-known anchors** that are extremely important for database modeling:

- countries;
- currencies;
- languages;
- days of the week;
- US states.

Each of those anchors has a well-known list of IDs that look like short strings.

For example, **countries** are represented by two-letter strings such as *"no"* for Norway and *"tr"* for Turkey. The complete list is standardized in the ISO 3166-1 alpha-2 standard.

Most common **languages** are represented by two-letter strings such as *"es"* for Spanish or *"ja"* for Japanese. They are defined in the ISO 639-1 standard. There are many languages, so some of them require three letters. Also, there is an aspect of regional

varieties, the most common of those being, perhaps, British English vs American English. This could be handled using the POSIX format for locales, such as *"es_AR"* and *"es_ES"* for, correspondingly, Argentinian dialect of Spanish and standard Spanish.

Currencies are represented by three-letter strings such as *"EUR"* for euro, or *"MXN"* for Mexican peso. A good practical list is probably the one in Wikipedia: `https://en.wikipedia.org/wiki/List_of_circulating_currencies`.

Days of the week have commonly accepted abbreviations: "mon", "tue", "wed", "thu", "fri", "sat", "sun". Using small strings may be preferable to numbers because the world is divided with regard to what the first day of the week is, Sunday or Monday.

Finally, a list of postal abbreviations for **the states in the USA**, such as "AZ" or "WY".

Here is an example definition of one such anchor:

Anchor	*ID example*	*Physical table name*
Country	*"pt", "fr", ...*	`countries`

As any anchor, those can have associated attributes. For example, you can define a human-readable country name:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Country	What is this Country's **name**?		
string	*"Portugal"*	`countries.name VARCHAR(64)`	

5.7.9 *Countries, currencies, and languages in your business*

Which countries, currencies, and languages does your business support? This may be a very important decision that has legal

and financial consequences. Because of that, you have to carefully maintain the list of countries, currencies, and languages that are specifically recognized.

On a physical level, this may be implemented as small lookup tables that contain some small number of rows. Adding new rows into such tables may be complicated and require careful planning. Why?

- Working with a new currency may require legal preparation and approval;
- Sometimes you may have to recognize some countries and territories in a specific way. For example, if you own an international delivery business, it may make sense to distinguish between Greenland and mainland Denmark.
- Supporting a new language can also require some non-trivial effort.

Sometimes you may need to maintain this list directly in the code. This would allow you precise control over what is supported by your application.

Note that this approach to IDs allows you to add application-specific records. For example, country codes starting with "x" are explicitly reserved for custom purposes, for example "xa". Also, you can use well-known currency codes such as "BTC".

5.8 HANDLING TIME IN THE PHYSICAL MODEL

Let's go back to a use case that we started to discuss in the "Handling time in the logical model" section above.

Let's repeat the brief problem definition. Imagine that our system sells tickets to concerts around the world. We want to store and display the door time, which is a local time in a specific city.

Now let's repeat the logical model filling in the information about physical storage.

Here is the list of **anchors**:

Anchor	*ID example*	*Physical table name*
City	1, 2, 3, ...	`cities`
Timezone	*"Europe/Brussels"*	Stored in tzdata (time zone database) NB: ID type in links would be: `VARCHAR(64)`
Concert	1, 2, 3, ...	`concerts`

We use straightforward names for the tables.

Note that the "Timezone" anchor is special: it is stored in the system time zone database, so there is no separate table.

List of attributes:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
City	What is the **name** of the City?		
string	*"Antwerp"*	`cities.name` `VARCHAR(64)`	
Concert	When do the **doors open**? (in city time zone)		

date/time	*"2024-10-31 18:30:00"*	`concerts.`
		`door_open_time`
		`DATETIME NULL`	

We use straightforward names for the columns and default physical types. Finally, a list of links:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Timezone : City	**1:N**	A City can have *only one* Timezone A Timezone can cover *several* Cities	`cities.timezone_id` NB: type is `VARCHAR(64)`
City : Concert	**1:N**	A Concert can happen in *only one* City A City can host *several* Concerts	`concerts.city_id`

Both links are 1:N, so we use table columns as usual. Note that the first link has an unusual column type: `VARCHAR(64)`. From the list of anchors we can see that the time zone ID looks like "*Europe/Brussels*".

Assembling all the data, we arrive at the following tables:

```
CREATE TABLE cities (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(64) NOT NULL,
  timezone_id VARCHAR(64) NOT NULL
);

CREATE TABLE concerts (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  city_id INTEGER NOT NULL,
  door_open_time DATETIME NULL,
  INDEX (city_id)
);
```

The table definitions are incomplete: for example, we certainly want to add a link between a Concert and a Band (or just the name of the band)? But the goal of this section is to focus on handling time (and time zones), so we skip irrelevant parts.

6

OTHER TABLE DESIGN STRATEGIES

6.1 TABLE DESIGN CONCERNS

There are four main concerns that underlie every table design strategy.

First, the result tables must be able **to store all the data defined in the logical schema**. This requirement may sound trivial, but sometimes people cut corners or get confused. As a result, database schema may just be incomplete. One way to make an incomplete database schema is to skip defining a logical schema.

Second, you need to understand **the effort needed to add a new attribute, link or anchor** to the database. Note that here we only talk about adding a new element, and not about other ways of database evolution (changing, deleting, etc.). Adding a new element is probably the most common operation, and it is expected to be roughly the easiest.

Third, we need to consider how the database would **handle the commonly expected read-only queries, performance-wise**. Queries could be classified in three groups:

- **Naturally performant.** For example, retrieving some attributes by a set of IDs is virtually always the cheapest possible operation, because it is naturally optimized in almost all databases.
- **Easily optimizable.** For example, if you need to query by a value of some attribute, you can easily add an index on the corresponding table column.
- **Specifically optimized.** Some queries are so important for the system performance that you need to optimize the database specifically for them. You need to be careful with

such optimizations. You can only optimize for one, maybe two most important queries.
- **Ad-hoc arbitrary.** Sometimes you have to query your data in a way that is not supported by any optimization, natural or specific. However, table design should support some unforeseen use cases, even if in a non-optimal way. Only very rarely you can completely disallow arbitrary queries.

Fourth, sometimes you need to consider **data insertion performance**. This is only relevant for the scenarios where the data is added about 10-100 times per second or more often. Some table design strategies are specifically targeted at this aspect. What happens if many clients are changing the data simultaneously? Is there a performance degradation or a delay?

The list of concerns is prioritized roughly from less advanced to more advanced:

- everyone is interested in decreasing the amount of administrative hassle needed to add a new attribute or a link;
- many systems would be more pleasant to use if they have a faster query response time; at the same time, it is practical to tolerate the fact that some queries would not be as fast as theoretically possible;
- only some systems truly need to worry about data insertion performance; very often the underlying database is already capable to handle the load;

6.2 IS THERE A RECOMMENDED TABLE STRATEGY?

First, this book does not give recommendations on choosing a table strategy. We avoid prescriptive statements like "choose table strategy X because it is . . . ". This is intentional.

On the other hand, we can reframe the statement in the following way: "If you choose table design strategy X, here are the most probable consequences". We are particularly interested in the consequences in the areas listed in the previous section:

- completeness of the logical schema implementation;
- complexity of adding a new anchor, link or attribute;
- handling read-only queries, specifically:

    - which of them are naturally performant;
    - which of them are easily optimizable;
    - which of them could be specifically optimized for;
    - how ad-hoc arbitrary queries are handled;
- behavior under a heavy data insertion load;

6.3 TABLE-PER-ANCHOR, REVISITED

We use a "table-per-anchor" in this book because it is most convenient for teaching, and is a good default for most small-scale systems.

If you choose table-per-anchor strategy, here are the most probable consequences:

- Logical schema is directly transferred to the physical design.
- Adding a new attribute or link may be easy or hard, depending on how much data there is in a table being changed. On the other hand, if there are many attributes in some anchors, the tables become inconvenient for people to use.
- Queries that use primary keys are naturally extremely performant.
- If you query by an attribute value, often you can easily optimize by adding an index. Note, however, that this only works up to a certain extent.
- You can optimize for specific queries by introducing secondary representations, see the chapter below.
- A common relational database will go out of its way trying to process any ad-hoc query that you throw at it. It may not always be successful, but historically a staggering amount of engineering effort went into finding any possible way to squeeze a bit of extra performance from the relational databases.
- As for the data insertion performance, common databases go to great lengths to optimize common scenarios. However, if you have a truly demanding system, some scenarios would require additional engineering effort.

Basically, table-per-anchor design strategy provides a useful baseline. We can discuss every other table design strategy comparing it with this one.

6.4 SIDE TABLES

In a serious system, you can find dozens or even hundreds of anchors. Moreover, in many businesses there are a number of central anchors that have lots of attributes.

Imagine a web application that helps users find a restaurant that they like and book a table. The underlying database would have a Restaurant anchor, and this anchor will certainly have many attributes. In a mature product we could have a couple hundred attributes, or even more.

Were we to use the table-per-anchor strategy, explained in the previous chapter, our "restaurants" table would have 200+ columns, and also probably a number of link columns. Such tables are awkward to deal with:

- people have a hard time browsing a list of 200 items that tend to be abbreviated and sometimes not very clear;
- adding a new column to a big table may be expensive, sometimes prohibitively so (see the "Table rewrite" chapter for discussion);
- some attributes are required be stored in a dedicated database that is managed in a special way; for example, all sorts of personally-identifiable data (PII) or finance-controlled data; sometimes there is a legal obligation to do so;
- narrower tables can have performance benefits (that is never guaranteed but it's possible);
- some logical attributes should just be stored in an entirely different storage; for example, image files (such as podcast episode covers) better be uploaded to a cloud object storage, such as Amazon S3;

For all those reasons, people often create side tables. They are designed in exactly the same way as anchor tables, but have a different name. They can be created in the same database instance, or in a different database.

Here is an example of a table that contains the users' personal information:

```
CREATE TABLE user_personal_data (
  id INTEGER NOT NULL PRIMARY KEY,
  legal_name VARCHAR(256) NOT NULL,
  date_of_birth DATE NOT NULL,
  citizenship VARCHAR(3) NOT NULL
);
```

As you can see, we've moved the user's legal name, date of birth, and citizenship information into a separate table. This table can even reside in a different database, or can be protected by stricter access control.

Note that here, we do not use the `AUTO_INCREMENT` keyword (speaking in MySQL dialect). This is because the definite list of user IDs would be maintained in the main anchor table, "`users`".

In PostgreSQL, similarly, the main anchor table would have the primary key defined as:

```
id INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY
```

But the side table would just be "`id INTEGER PRIMARY KEY`".

6.4.1 *Naming and composition of side tables*

So, the attributes of a large anchor can be distributed between the main and several side tables. Two questions arise:

- how do we name those side tables, and
- how do we decide which attribute goes where?

The answer is that it's up to you. You have to choose an organizing principle of some kind and stick to it. Don't worry, no matter what you decide, sooner or later you will run into an attribute that defies your system and will make you hesitate before making the decision. This is the Game of Tables, described below.

Here are some commonly used organizing principles:

- **topical**: you may have separate side tables for cuisine-related attributes, for finance-related attributes, for contractual information, and so on;

- **frequency of use**: you may have a "core" set of attributes that are used in many places; the ones that are used less often would go into an "extra" set of attributes, or into "details";
- **department-based**: side tables containing the information that is maintained by a single department or a team;
- **regulatory-driven**: sensitive information such as personal or financial data go into a separate table, probably in a separate database;
- **technology-driven**: for some attributes, technical considerations are so important that you cannot help but to store them in a database using a different technology; one example already mentioned is storing images in a cloud-based object storage;

All those methods are fuzzy and subject to occasional bargaining. Sometimes it becomes clear in retrospect that an attribute should have been stored in a different place. In more important cases you will even have to do a database migration.

6.4.2 *A stopgap table*

There is one more possibility that may occasionally be useful in practice. Sometimes you need to introduce an attribute (basically, a table column) into a running system, but you're very much constrained by the urgency of this change. There would be a very important reason for this change, and the clock will be ticking. (You will recognize this situation when you find yourself in it.)

You may not be able to add a new column into any of the existing tables, because it would be hard or impossible for some technical reason. Here is a tried and tested solution: you can just create a new side-table table, with only this particular attribute. Your table will basically contain just two columns: an anchor ID and an attribute value.

Creating new tables is essentially **zero-cost** in virtually all database management systems. It is also **instant**, and that would be the decisive factor in your situation.

6.5 JSON COLUMNS

Some anchors can have a bunch of attributes that you, as the database designer, do not particularly care about. That is, you need a way to store many attributes, but you don't want to adjust the physical table schema each time a new attribute is added.

Suppose that you're building a website for restaurant reservations. You want to give your users maximum flexibility in finding a suitable dining place. For that, you want to keep the detailed information about restaurant properties, for example:

- *Is it pet-friendly?*
- *Does it have a kids play area?*
- *Is it vegan-only?*
- *Is it halal?*
- *Is it wheelchair-accessible?*
- *Does it have a terrace?*
- *Is it on the rooftop?*
- *Is it romantic?*

You can imagine many more different attributes. Traditional table-per-anchor design strategy requires you to create a new column for each of those attributes. Technically it's possible, but the table could quickly become too wide. Working with such a table may become inconvenient. Also, adding a new attribute would require a little bit of database administration, which may also be non-trivial technically (see "Evolving your database" chapter for details).

One solution is to define a column that contains JSON object:

```
CREATE TABLE restaurants (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  -- other columns
  extra_attributes JSON NOT NULL DEFAULT ('{}')
);
```

We use the modern JSON physical type here. For older databases something like TEXT can be used. Explaining how to work with JSON columns is outside of the scope of the book. You need to consult the documentation for your database.

The objects that would be stored in such a column would look like this:

```
{
  "is_pet_friendly": true,
  "is_wheelchair_accessible": false,
  "is_rooftop": true
}
```

JSON supports simple types: strings, boolean values and numbers.

To start using another attribute, you add it to the logical model:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Restaurant	How many **Michelin stars** does this Restaurant have?		
integer	*1, 2, 3*	`restaurants.extra_attributes: $.michelin_stars` *JSON integer*	

Take a look at the "Physical column" column:

```
restaurants.extra_attributes: $.michelin_stars
```

First we provided the name of the table and the JSON column: "`restaurants.extra_attributes`". Then we specify the JSON key name, using the standard JSON Path notation: "`$.michelin_stars`". Dollar sign here means "object root level", and the "`michelin_stars`" is the JSON key to use.

Here is how the JSON value looks like (together with one other random attribute):

```
{
  "is_vegan": false,
  "michelin_stars": 1
}
```

Note that we've specified "*JSON integer*" as the physical type of that attribute.

Another example of using JSON columns could be found in the "Books and washing machines" chapter below.

7

DEALING WITH ABSENT DATA

Let's begin with attribute values as the most common case. As usual, we start with the business requirements.

Suppose that you run an e-commerce system selling dresses, and one of the most important properties of a dress is its color. Sooner or later you will find that dress color is not always available. One common case is that the data is imported from a third-party provider which just does not give you this information. You can't make them fix their data. How to deal with this?

Here is another example: in many social networks, there is usually a text area called "bio" where the user can enter any information about themselves. Of course, many users don't bother writing anything there, how do we deal with that?

Finally, here is another common important scenario. Imagine a serious online movies forum. Some day as a matter of policy all users are required to disclose what their favorite movie is. Without this information their access to the forum is restricted. However, in some jurisdictions they actually have a legal protection of their right to not disclose their favorite movie. So the users should be able to state that they refuse to provide this information, and our forum must honor it. How do we handle this?

7.1 USE CASE: USER'S BIO

Let's begin with the simplest case: "bio". Here is the attribute definition:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
User	What is the User's **bio**?		
string	*"Movie-goer with almost 5 years of experience"*	⟨TBD⟩	

Using the table-per-anchor table design strategy, we can design the following table to store the information about user (including a "date of birth" column to make it a bit more realistic):

```
CREATE TABLE users (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  date_of_birth DATE,
  bio VARCHAR(1000)
);
```

Suppose that we have a user with *ID=10*, whose date of birth is *2001-02-03*, and who did not bother to provide anything in the bio during signup. In the table-per-anchor case, we must write something into every column. What do we put into the "`bio`" column?

We don't have any real attribute value, so we must use some fake special value. This sort of value is called a **sentinel value**. In this particular case we have two commonly used options: NULL and an empty string.

7.2 SENTINEL VALUES: NULL

The most common sentinel value in relational databases is NULL. It can be used for virtually all data types: strings, integers, decimals, dates, date/time, binary blobs, etc., etc.

The idea of a NULL value is a topic of decades-long historical debate. Discussing the semantics of NULL values is out of scope for this book, so I'm going to refer you to books and tutorials about SQL.

Two facts are indisputable, though. First, NULLs are present everywhere in real-world databases and there is nothing wrong about them. Second, NULL is just one of possible sentinel values.

For each column you can specify if it's allowed to have NULL values, or every value must be non-NULL.

Let's look at the definition of the "`users`" table above. The first column is defined as:

```
id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT
```

The "`id`" column in the anchor tables must be marked as "`NOT NULL`", because it is marked as "`PRIMARY KEY`". Why? Because having an undefined ID of an anchor does not make sense.

Now let's look at the "`bio`" column:

```
bio VARCHAR(1000)
```

By default, an attribute column can contain NULLs. You can also explicitly mark it as such (for readability):

```
bio VARCHAR(1000) NULL
```

7.3 OTHER SENTINEL VALUES

In some cases, you may have additional options to represent the absence of attribute value. In the "bio" case discussed here, you can use an empty string:

```
bio VARCHAR(1000) NOT NULL DEFAULT ""
```

Here, we defined the "`bio`" column as "`NOT NULL`", but we immediately chickened out and said that if the value is not provided then it's going to be an empty string by default.

For many string attributes this is a very common approach that you can see in many real-world databases.

However, for other physical types there may not be a very good sentinel value that is not a NULL.

For example, there is no "special integer number". Zero value could be such a sentinel value for some integer attributes, but not for others. NULL here clearly and unambiguously shows that there is no real integer number.

For monetary values you certainly want to be extra careful. Zero is not a good sentinel value, because there are valid zero

monetary amounts: for example, a tax on some items can be 0.0%.

For dates and times, sometimes you can use a special value *"0000-00-00 00:00:00"*, if your database supports it. But again, maybe you would better use a NULL here.

One interesting case is yes/no attributes. Logical yes/no type has exactly two values: *"yes"* and *"no"*. But if the yes/no attribute is stored in the NULLable column then it's as if you had three options:

- *yes*;
- *no*;
- *not provided*.

This case is one of the big reasons underlying the decades-long debate mentioned above. It is recommended that you avoid NULLable columns for the yes/no case.

Same problem exists for the either/or/or attributes. Logical either/or/or type has exactly N values. Storing it in the NULLable column adds one more implicit status: *"not provided"*, which may be confusing. It is recommended that you avoid NULLable columns here too. If needed, you can just add another explicit status (e.g., *"not_provided"*) into your type definition.

7.4 SENTINEL VALUES EXIST ONLY ON A PHYSICAL LEVEL

On the logical level there are no sentinel values. This is because **sentinel values are only needed for certain table design strategies**, particularly for the table-per-anchor approach.

Imagine that we decided to create a side table specifically for storing user bios. This would make total sense for some kinds of attributes. Our table would look like this:

```
CREATE TABLE user_bios (
  id INTEGER NOT NULL PRIMARY KEY,
  bio VARCHAR(1000)
);
```

Here, you don't need any sentinel values. If you have a row in such a table it means that the attribute value is set for a certain ID. If there is no row then the bio was not provided by the user. So, we can add a "`NOT NULL`" modifier:

```
CREATE TABLE user_bios (
  id INTEGER NOT NULL PRIMARY KEY,
  bio VARCHAR(1000) NOT NULL
);
```

and make sure that when the user removes their bio text completely, we delete the entire row.

Another example where sentinel values are not needed is the JSON-based table approach. If the user did not provide a bio, you can just not set a corresponding JSON key. (Additionally, of course, JSON provides its own "`null`" value which is different from the SQL-level NULL.)

7.5 EXPLICIT REASONS FOR MISSING DATA

In some cases, you want to know exactly why a certain attribute value is not available. The most important use case for that is legal requirements. Let's take a look at the business requirements presented above.

> *"Imagine a serious online movies forum. Some day as a matter of policy all users are required to disclose what their favorite movie is. Without this information their access to the forum is restricted. However, in some jurisdictions they actually have a legal protection of their right to not disclose their favorite movie. So the users should be able to state that they refuse to provide this information, and our forum must honor it. How do we handle this?"*

We can see that there are three possible states: a) pending: for the users who did not login yet after the new policy was introduced; b) submitted: for the users who submitted their favorite movie information; c) refused to answer, citing their legal rights. In "pending" and "refused to answer" states there is no favourite movie information. We'll model this using two attributes:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
User	Did the User provide their favorite movie information?		
either/or/or	*"pending"* *"submitted"* *"refused_to_answer"*	⟨TBD⟩	
User	For the users who submitted their **favorite movie**, what is its **name**?		
string	*"Jaws"*	⟨TBD⟩	

The second attribute is tangled with the first. Its value only makes sense when the value of the first attribute is *"submitted"*.

Note that the questions are worded rather informally: one could argue that *"did the User . . . ?"* implies a yes/no response. However, the intention is clear enough, given the list of possible either/or/or values. In the end, if you are asked *"Did you like their breakfast?"* you could respond *"I did not have time for breakfast when I stayed there"*.

Physical definition of two attributes does not need to be fancy. If you use a table-per-anchor table design strategy, the hardest part may be to find a good column name for the first attribute. Let's try:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
User	Did the User provide their favorite movie information?		

either/or/or	*"pending"*
	"submitted"
	"refused_to_answer"
		`fav_movie_status`
		`VARCHAR(24)`
		`NOT NULL`
		`DEFAULT "pending"`	

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
User	For the users who submitted their **favorite movie**, what is its **name**?		
string	*"Jaws"*	`fav_movie_name VARCHAR(256)`	

As a SQL table definition:

```
CREATE TABLE users (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  -- other attribute columns skipped
  fav_movie_status VARCHAR(24) NOT NULL DEFAULT "pending",
  fav_movie_name VARCHAR(256)
);
```

When a row is inserted into the "`users`" table, the "`fav_movie_status`" column is set to *"pending"*, and the value of "`fav_movie_name`" is `NULL`. It's fine, because we cannot use this name until the status becomes *"submitted"*.

You have to carefully write your SQL queries so that you only access the movie name information if it is in the right state.

7.6 SUMMARY

On a logical level, an attribute can either have a definite value, or be unset. Generally speaking, there are no "special" attribute values on a logical level.

When we move to the physical level, we often need a special value, to be used if there is no real value available. The term for that is *sentinel value*. The most common sentinel value is NULL. For some physical types it's possible to use other sentinel values, such as an empty string.

For some table design strategies, sentinel values are not needed, because the absence of value is represented structurally. Usually it's the absence of a corresponding table row, or a JSON key.

The concept of NULL was a subject of extensive historical debate, so be prepared to learn multiple contradictory opinions about that.

Sometimes you need to know the reason why exactly the value is missing. One important example is legal requirements related to some information. You need to model this explicitly, using tangled attributes.

8

SECONDARY DATA

Anchors, links and attributes together are considered primary data. In other words, they are the source of truth.

For example, when we store the information about an order made by a customer, it is stored in anchor tables, attribute columns and link tables. If we lose this information, for example a table row is accidentally deleted, the system would think that this is how it is. If we delete one item from the order, we'll send a package without this item. If we delete information about the order, the system won't recognize the order number. This is primary data. If it gets deleted accidentally, you need to restore from backups.

There is another sort of data: secondary data. This is the data that was copied from the primary data, duplicated, reorganized, flattened, post-processed and so on. There are many ways to introduce and use the secondary data. If it gets deleted accidentally, it can be recalculated from the primary data (theoretically, at least).

Here is an incomplete list of things that we consider secondary data:

- cached (pre-computed) columns;
- pre-aggregated tables;
- denormalized columns and tables;
- ML models generated and updated based on data;
- materialized feeds;
- flat tables;
- full-text indexes;
- copies of data in different databases;
- data caches such as Memcached or in-memory Redis;

- ordinary database indexes have many characteristics of secondary data, if you squint hard enough;
- etc., etc.

We'll discuss the following aspects of secondary data:

- **secondary data is always introduced for performance or convenience**;
- **secondary data is always ad hoc (introduced for a specific purpose);**
- **secondary data is never free**. We pay for the performance or convenience with extra effort and resources that are needed to maintain the secondary representation. It's extra storage, extra CPU processing time, extra software development effort;
- **secondary data lives at the physical level**. Secondary data is NOT an attribute, an anchor or a link. It is a table, column, or an index of some kind;
- **secondary data may not be immediately obvious**, so it needs documenting;
- **secondary data always has a propagation delay**, even if sometimes it's effectively zero;
- **secondary data can get abandoned**;

Unfortunately, in-depth discussion of common approaches to secondary data is out of scope of this book. What we're going to do instead is:

- discuss a simple example: cached column;
- discuss the general aspects of secondary data, using cached column as an example;
- talk a little bit about other types of secondary data;
- discuss documenting secondary data.

8.1 CACHED COLUMN EXAMPLE

Here is a simple example: suppose that we have a social network. Users can write posts. It's easy to see two anchors: User and Post; one attribute: text of the post; one link: User writes Posts. Now suppose that we have a user page, for example `https://social.example.org/thomas`. On this page we want to

show some user information, including the number of posts that they have written. How do we fetch this data when rendering the page?

We would probably use the following query:

```
SELECT COUNT(*) FROM posts WHERE user_id = 123
```

We will execute this query every time the page is accessed. This is not a super-heavy query, but what if we have a lot of traffic and we want to optimize?

Well, we could create a column in the users table, called "`posts_count`". This column would contain an integer number: how many posts were written by the user. Initially this number is zero. Every time a user writes a new post, this column is recalculated; the same thing happens when they remove a post.

When we have this column, we can use a much more efficient query:

```
SELECT posts_count FROM users WHERE user_id = 123
```

8.2 THERE IS NO FREE LUNCH

The cost of this improvement is that we need to maintain this column. We have to add the code that recalculates this column, and add it in all the appropriate places.

If there is a bug in our application, the user can write a new post but the post count column would not be updated. Users may notice and complain. We would have to write extra code that cross-checks the secondary data against the primary data and refreshes the secondary data accordingly.

Note that we have introduced this column for a very specific reason: we wanted to optimize rendering of the user's page. If we did not have this page, or if we did not need to show the number of posts on this page, we wouldn't need this column at all!

Also, we assumed that we have a lot of traffic and we want to optimize. What if the user page is just rarely accessed and we are fine with the generation speed? In that case we do not need to introduce this column, and just use the first query directly.

This is a simple example, but it demonstrates everything that may routinely happen with every piece of secondary data. In this tiny example we've seen that:

- secondary data is introduced to improve performance;
- it needs extra storage space;
- it needs some amount of extra computing resources for updating;
- on the other hand, it saves some amount of computing resources when this data is actually used;
- it was introduced for a particular reason;
- it requires extra developer attention;
- it can get out of sync with the primary data because of the bug, or an outage;
- it requires a procedure to fix the discrepancies when they inevitably arise;
- it's possible that the secondary data outlives its usefulness, and may be decommissioned;
- secondary data routinely gets forgotten and abandoned, and can waste resources for years.

This list is true for any sort of secondary data.

8.3 CACHED COLUMN IS NOT AN ATTRIBUTE

Proper attributes can be changed, at least theoretically. In some cases this never actually happens in practice. For example, suppose that we have parcels that have tracking numbers. Tracking number is an attribute of the Parcel anchor. Tracking numbers never change in practice, but they **can** be changed in principle.

The *"number of posts written by user"* is not an attribute because it cannot be changed directly, in principle. The only way to change this value is to write more posts, or delete some posts. After that the value will get recalculated.

8.4 DISCOVERING SECONDARY DATA

We continue the discussion of our cached column example. One aspect of secondary data is that it may not be immediately obvious that it exists in the first place.

Suppose that we joined the company that develops the social network that we discuss. Our first task is to find the top 10 most active users, in terms of the number of posts they wrote.

There is not a lot of documentation, as usual. Or maybe there is, but we're in a hurry. We know that calculating this would be easy, because the "`posts`" table is so straightforward. So we just use the obvious SQL query:

```
SELECT user_id, COUNT(*) AS cnt
FROM posts
GROUP BY user_id
ORDER BY cnt DESC
LIMIT 10;
```

This query will work perfectly. But if we knew about the "`users.posts_count`" column, we'd probably started with the following query:

```
SELECT id, posts_count
FROM users
ORDER BY posts_count DESC
LIMIT 10;
```

which should be much more performant. But the point is that it's not guaranteed that we would even be able to find this. There is no easy solution for this, but some documentation may help.

9

EVOLVING YOUR DATABASE

Inevitably, sooner or later in your project's lifecycle, you will have to change the structure of your database.

There are several practical reasons for the database to evolve:

- normal development of **new features**;
- changing the definition of existing database elements: anchors, attributes and links. This happens when the **business requirements change** in such a way that the original implementation is incompatible with them.
- improving database performance or scalability by **reorganizing physical schema**;
- introducing secondary representations, such as **derived columns and pre-aggregated tables**;
- **improving organizational scalability**: for example, making it easier to add new attributes by introducing a JSON column;
- **removing data** that is no longer needed, or no longer wanted;
- **migrating to a different database** for performance reasons, compliance reasons or for organizational scalability;
- **building data warehouses**: derived representations of primary data for better reporting, data analytics and so on.

Any database evolution requires changing three things:

- logical schema;
- application code;
- physical database itself.

Logical schema is just a document, so changing it is extremely cheap: you just edit the document. The effort needed to change the application and the database schema may vary significantly.

9.1 ELEMENTARY DATABASE MIGRATIONS

Every database migration consists of elementary pieces. In this book, we're going to treat database migrations incrementally. We'll discuss each smallest type of change separately (for example, adding an attribute or removing a link). In practice, sometimes you bundle together several such changes, for example you introduce a new anchor, several attributes and a link in the same database migration. But most changes would just naturally be handled piece-by-piece: it's often also less risky.

Let's write down all elementary database migrations. Three big categories are: adding something, removing something and changing something.

So, first we have:

- adding new anchor;
- adding new attribute;
- adding new link;

Sometimes we want to remove some stuff:

- removing a link;
- removing an attribute;
- removing an anchor;

Changing the definition of something:

- changing the physical type of anchor ID;
- changing the physical type of an attribute;
- changing the semantics of an attribute;
- changing the definition of either/or/or attribute;
- changing the cardinality of a link (from 1:N to M:N or vice versa);
- changing table design strategy for an attribute;

We started discussing the secondary data in the previous chapter, so let's list related migrations here:

- adding new derived column;
- removing derived column;
- adding new pre-aggregated table;
- removing pre-aggregated table;

- changing the definition of pre-aggregated table;

Also, you sometimes have to do strictly physical migration that would not be reflected in the logical schema at all:

- adding/altering/removing an index;
- changing table storage mechanism (such as adding compression or sharding);

Systematic discussion of every bullet point is out of scope for this book, but we'll cover one important case: adding an attribute. It demonstrates an important concern behind table design: how much effort is required to add a new element.

9.2 TABLE REWRITE

But before doing that we need to understand something that is called table rewrite. Table rewrite happens on a physical level, and it is a major driving force behind database design and database migrations planning.

Imagine a database with a very simple table called "`users`" with two columns:

ID	*Name*
1	*"Alice"*
2	*"Bob"*
3	*"Carol"*
...	...

Suppose that the table contains a lot of rows like that, say 10 million. This table serves a website where new users constantly get registered, existing users decide to change their displayed name, and sometimes they decide to delete their account.

Suppose that this website runs a classic relational database, such as MySQL or PostgreSQL. In this environment table rewrite is clearly visible and understandable.

Now suppose that we want to add a new column to this table: user's day of birth. For existing users, this value will be set to `NULL` (it doesn't matter for our purposes). After this change we want our table to look like this:

ID	*Name*	*DOB*
1	*"Alice"*	`NULL`
2	*"Bob"*	`NULL`
3	*"Carol"*	`NULL`
...	...	...

We have the same number of rows, around 10 million.

What happens physically here? Let's grossly (and I mean *grossly*) oversimplify. Each table is contained basically in a separate file on disk. For the sake of illustration, we could pretend that the file is just a comma-separated file, with each field having a fixed width. Say, 10 bytes for the ID and 10 Latin characters for the name. If the name is shorter, it is padded with space characters. The fields are separated by commas for readability, and there is also a newline character between the rows.

Of course, no real database has this sort of physical data layout, but it's still useful to understand the concept of table rewrite.

So, here is how our file would like before the change:

```
0000000001,Alice␣␣␣␣␣\n
0000000002,Bob␣␣␣␣␣␣␣\n
0000000003,Carol␣␣␣␣␣\n
...
```

We have 10 + 1 + 10 + 1 bytes per row, so if we have 10 million rows that means that our file would be around 209 megabytes long.

After we change the table structure, we want our file to look like this:

```
0000000001,Alice␣␣␣␣␣,NULL␣␣␣␣␣␣\n
0000000002,Bob␣␣␣␣␣␣␣,NULL␣␣␣␣␣␣\n
0000000003,Carol␣␣␣␣␣,NULL␣␣␣␣␣␣\n
...
```

(We want to have space for, say, *"1948-02-04"*, so we allocate another ten bytes for the date, or for the string "`NULL`"). After this change, each row would take 10 + 1 + 10 + 1 + 10 + 1 = 33 bytes. The entire file would be around 314 megabytes.

The only reasonable way to go from one to another is to create the second file from scratch, and copy all the records from the original file. After this is done, we can delete the original file and replace it with the new file. This is exactly what table rewrite is.

What's the problem here? The problem is that our website keeps running. New users get inserted, they change their names, sometimes they delete their accounts.

Copying a 314 megabytes file is not an instant operation, it may take seconds, maybe even tens of seconds. What if we copied a record for Alice, but at the same moment he decided to change the name to *"Cooper"*? Do we need to go back to the beginning of our new file and change it also? Many other scenarios are possible.

Classic relational databases made a rational decision to avoid all this complexity. When the database administrator requests that the table structure be altered, the database blocks all incoming update requests for that specific table, such as INSERT, UPDATE and DELETE. Also, in many cases SELECT queries are also blocked. Database server rewrites the table file, and then proceeds with all the pending requests.

But how long would that service interruption last? 314 megabytes is not a lot: your table can have tens and hundreds gigabytes of data. So the time could be measured in minutes, tens of minutes, hours, and many hours. Remember that the database server only blocks queries related to that table, but the other tables usually stay available. That means that they compete for the disk bandwidth with the table rewrite process, and this can unpredictably increase time needed for the rewrite.

There is another concern that we have with table rewrite: disk space. Even if we can tolerate the service interruption time, we still need the disk space to hold a copy of a table file. Do we have 314 megabytes free? We may even find ourselves in a tricky situation where we want to remove an unused table column to save disk space, but we cannot do that because there is no disk space!

Many interesting solutions have been developed to deal with table rewrite. We're going to discuss them later in the book. Now let's go back to the general question of database migrations.

9.3 ADDING AN ATTRIBUTE

Adding an attribute is a perfect elementary migration to discuss first. Suppose that our system uses one simple anchor: *User* and one simple attribute: name of the User. Here is the excerpt from our logical model, starting with anchors:

Anchor	*ID example*	*Physical table name*
User	1, 2, 3, ...	`users`

Attributes:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
User	What is the **name** of the User?		
string	*"Matthew"*	`users.name VARCHAR(128)`	

When we built this system, we used the table-per-anchor table design strategy. Thus, our users table looks like this:

```
CREATE TABLE users (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(128) NULL
);
```

In a real table there would probably be many more columns, but we keep it short for brevity.

A new system requirement appears: we need to gather user's dates of birth. Here is our plan:

- Step 1: Update logical model;
- Step 2: Run database migration;
- Step 3: Update the code of our system to use the new attribute;

9.3.1 *Step 1: Update logical model*

Here is the updated logical model, with the proposed attributes below:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
User	What is the **name** of the User?		
string	*"Matthew"*	`users.name` `VARCHAR(128)`	
User	What is the **date of birth** of User?		
date	*"1974-07-30"*	`users.day_of_birth` `DATE NULL`	

Note that this step is very cheap: you just edit a document with the logical model. We'll appreciate this more when we're going to discuss more complicated features that require designing several new anchors, attributes and links.

Had we known about this attribute from the beginning, our initial table would look like this:

```
CREATE TABLE users (
  id INTEGER NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(128) NULL,
  day_of_birth DATE NULL
);
```

But we already have a table, and so we need to alter its definition. For that, the following SQL command would be needed:

```
ALTER TABLE users
ADD COLUMN day_of_birth DATE NULL;
```

Teaching SQL operators comprehensively is outside the scope of this book; you will have to pick up a good book on SQL, or read the documentation for your database server.

9.3.2 *Step 2. Run database migration*

There are many software tools to run database migrations in a controlled way. Your development framework probably has one, and you should really use it.

But for the sake of simplicity, let's pretend that we can just connect to the database server and directly execute the ALTER TABLE statement. You probably begin to suspect that the previous chapter, "Table rewrite", has something to do with this idea.

Yes, this is a typical example of a physical schema change that would cause table rewrite. In our toy example it would probably be unnoticeable, but I still want you to learn to pay attention to this. Later in the book we're going to talk about other table design strategies that can help you to deal with table rewrite.

9.3.3 *Step 3. Update code*

After we've successfully executed the previous step, we can now use the new column. For example, we can ask the new users to provide their day of birth during registration; for existing users we can show some sort of banner that invites them to some page where they can update their information.

So, you can just change the code of your application and deploy it as usual.

Note that we made the new column nullable (its physical type is `DATE NULL`). It means that our attribute would be unset for all existing rows. Also, for the newly-inserted rows, if your application does not provide a specific value, then it's going to be unset. This allows you to decouple database migration from updating the code. This decoupling is extremely important for many systems, and we're going to pay a lot of attention to keeping this separation.

9.4 DEALING WITH TABLE REWRITE

Several possibilities are available in practice.

First, we can just **go ahead with the ALTER TABLE** if our system can tolerate the estimated interruption time. Maybe our

table is just small, and the rewrite time would be just a fraction of a second. Maybe the table is used by a component that can actually tolerate a very long interruption. Maybe our system has some sort of maintenance window that we can use for this operation.

Second, sometimes it is possible to change table schema without interruption, using the **"online schema change" approach**. In the previous section we asked a rhetorical question: "What if we copied a record for Alice, but at the same moment he decided to change the name to "Cooper"? Do we need to go back to the beginning of our new file and change it also?" Well, it turns out that you can go all-in and implement a tool that handles all possible scenarios. This has been a game-changer for some larger installations.

Third, we've been talking about the classic relational databases. But we can design a database server that is just implemented in a different way. For example, **a physical data layout could be way smarter**, and could just handle schema change without any interruption? In some databases, this is exactly what happens.

Fourth, why do we even need to add a column to the existing table? What if we created another table especially for storing this particular attribute? Creating new tables is basically free in all known database systems. Our instinct to add a column to the existing table is generally valid. This is what we called "third normal form table design strategy" above. But **other table design strategies are possible**, and we're going to talk about them later.

I'm going to provide an overview of possible options that people decide to use. This is not advice, it's just a list of possibilities. We will discuss them in detail later. So, here you are:

- create a side table called "`users_dob`"; creating a new table is essentially free in all known database systems;
- use a JSON-based column, it allows to use arbitrary keys without the need to change table schema at all;
- use an approach called Entity-Attribute-Value (EAV), it allows to add new attributes without physical schema change;

None of those options are “free”, there are consequences for using them. We will discuss them in detail later in the book.

10

MOVIE TICKETS: REPEATED SALES PATTERN

Let's discuss how to model selling movie tickets. This use case illustrates a very common pattern that I call repeated sales pattern. We will only focus on the logical model. We're only going to talk about essential parts of this pattern, omitting some details.

"Repeated sales" pattern is obviously important to model many businesses: not only movie tickets, but also flight and train tickets, hotel reservations, cargo transportation and other examples. Outside of sales, this pattern arises in a well-known textbook case of "course assignments" that is often used in teaching database modeling.

10.1 BUSINESS REQUIREMENTS

Suppose that our business is a movie theater, with multiple screens. Inside every auditorium there are a number of seats, physical chairs. We want to sell tickets to people willing to occupy those chairs for a few hours at certain times of a specific day.

Our movie theater also buys rights to show various movies, as they are released.

Every day there are multiple showtimes at each auditorium. Anyone can buy a ticket for a certain time of the day at a certain date, when a certain movie will be shown.

For simplicity, we assume that every movie ticket costs exactly the same, no matter what time slot, movie, or a screen. It's not hard to add this improvement on top of the basic logical model we'll build.

Moreover, we won't even bother with payments. Generally speaking, there are several ways how you can get a ticket:

- a normal purchase;
- a replacement ticket as a result of customer service resolution process;
- membership card that allows, say, unlimited visits;
- winning at the movie quiz, etc., etc.

We won't be modeling any of that. Let's pretend that the ticket was somehow issued to the customer. Again, adding all of this later is not hard.

10.2 PER-DEPARTMENT MODELING

So, should we now start listing the anchors that we see in the business requirements? I see tickets, screens, seats, movies, showtimes, what else?

Let's pause for a moment and reconsider how we're going to build the complete model. It may be tempting to begin with tickets, because that is probably the most interesting business object. This is what the entire business is about.

But practice shows that when you work from scratch, starting from the "top-level" object may quickly lead you into confusion. There is a chance, of course, that you'll manage your way out, but I'd like to propose a more reliable approach. Its benefit is that it would help you make sense of the model that is more complicated than you can keep in your head.

Let's think in terms of business departments. Even if your company is very small, there are still teams of people tasked with different areas. In our example, there are:

- people who built and maintain the venue: auditoriums and physical chairs;
- people who decide which movies to buy;
- people who design the movie schedule for the upcoming days;
- and finally, people who sell tickets.

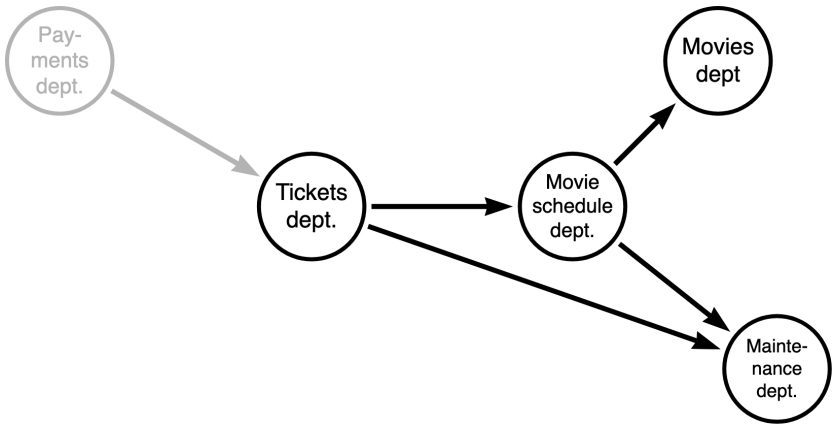
Those teams may closely work with each other, but their areas of responsibilities are pretty well-delineated. It helps with modeling if you think about each department independently.

Departments depend on each other. The tickets people cannot sell tickets without a schedule. The schedules department needs

to know what movies are available, their run time, and which screens are open for business.

But some departments do not have dependencies. People who buy movie rights need only to know their budget. The building maintenance department, in a sense, does not care about the movies: their job is to make sure that the auditorium is cleaned and safe. They can also sometimes engage in building a new auditorium, but again — they don't care about specific movies, time slots, tickets, etc.

Here is a dependency graph between the departments:



Arrow direction shows that a department depends on another department. We also show the "payments department"; we're not going to use it here, but if we were to model the payments part we'd just handle this department too.

We'll do the modeling from the right side, starting with the independent departments.

10.3 MOVIES DEPARTMENT

For the purposes of this chapter, we'll model a bare minimum of information.
Anchors:

Anchor	*ID example*	*Physical table name*
Movie	1, 2, 3, ...	—

And attributes:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Movie	What is the **name** of the Movie?		
string	*"Jaws"*	—	
Movie	What is the **running time** of the Movie? (in minutes)		
integer	*124*	—	

That's it. In a real department handling movies there would be a lot more information: contracts handling, outgoing payments to the studios, more information about the movie, etc. etc.

10.4 MAINTENANCE DEPARTMENT

This department is responsible for the venue maintenance. They ensure that auditoriums are cleaned and can be used safely. They care about the physical chairs: are there any broken chairs that are temporarily not available? Are some of the screens temporarily closed for renovation or a movie projector maintenance?

Anchors:

Anchor	*ID example*	*Physical table name*
Auditorium	1, 2, 3, ...	—
Chair	1, 2, 3, ...	—

We can confirm the anchors by using counting and adding sentences:

- *"there are 15 screens in our multiplex"*;
- *"there are 1200 chairs total installed at all the auditoriums"*;

And attributes:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Auditorium	What is the **name** of the Auditorium?		
string	*"Hall 3"*	—	
Chair	What is the **row number** of the Chair?		
integer	*3*	—	
Chair	What is the **seat number** of the Chair?		
integer	*8*	—	

We won't model here the following scenarios:

- broken chairs that you cannot sell tickets for;
- temporarily closed auditorium;
- new screen that is about to be opened in a couple of weeks, but not available yet;
- renovated auditorium so there is now a different seating plan.

But we must keep those scenarios in mind, they are important for the discussion of "repeated sales" pattern. We need to design

the basic structure of the database in such a way that those scenarios are possible to implement.

Ah, we need a link between the chair and auditorium:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Auditorium : Chair	**1:N**	An Auditorium can have *several* Chairs A Chair can be installed at *only one* Auditorium	—

10.5 MOVIE SCHEDULE DEPARTMENT

This department depends on the data from the other two departments. To construct a movie schedule, one needs to know:

- which movies are available;
- which auditoriums are open.

We won't go into details of how the movie schedule department lays out the schedule. They take into account holidays, movie premieres, box office returns, etc., etc. This is way out of our pay grade, let's just give them a tool to store their final result.

So, for every day in the upcoming couple of weeks, for every screen there would be a number of time slots. Generally speaking, the time slots may vary day by day, depending on the running time of movies being shown. For example, here is a list of timeslots, an excerpt from a movie schedule:

- *2025-01-18 at 10:00, Hall 3, "Jaws";*
- *2025-01-18 at 10:15, Hall 5, "Back To The Future";*
- *2025-01-18 at 14:00, Hall 3, "Casablanca";*
- ... and so on.

Anchors:

Anchor	*ID example*	*Physical table name*
Timeslot	1, 2, 3, ...	—

And attributes:

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Timeslot	What is the **date/time** of this Timeslot?		
date/time (local timezone)	*"2025-01-18 10:00:00"*	—	

There are no more attributes for the timeslot, but there are two links:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Auditorium : Timeslot	**1:N**	An Auditorium can host *several* Time slots A Time slot can be assigned at *only one* Auditorium	—
Movie : Timeslot	**1:N**	A Time slot can show *only one* Movie A Movie can be shown at *several* Time slots	—

That's it. Note that there is no mention of chairs anywhere. That's because chairs only become relevant when we talk about tickets, and tickets are handled by a different department.

10.6 TICKETS DEPARTMENT

The job of a tickets department is to sell the right to sit in certain chairs during a certain time slot.

This phrase may be awkward, but it brings an important distinction between a) chair and b) a ticket for this chair at a certain time. The problem is that in informal speech we would often use the same word for both. For example, you can say *"I booked a loveseat at the cinema for our date"*. But a person working at the maintenance department can say *"We ordered a replacement loveseat for the Hall 5"*. And the word "loveseat" means two different things here.

It's possible that you, dear reader, did not have this confusion in the first place. In this chapter we're spelling this out because it seems to be a common confusion that happens sometimes, and we want everyone to be on the same page here.

So, for each time slot we need to offer all the chairs in the corresponding auditorium for sale.

Here is another quirk of informal language that may be useful to understand. The word "sell" has two different meanings. *"I sell apples for $10 a piece"* does not mean that anybody is buying them.

Another sentence, *"I sold 20 apples for $8 a piece (with a discount)"* does not contradict the first one! We wanted to sell at one price, but we had to sell at a different price, that's normal. We may even be lucky and sell them for the offer price, but it's important to notice this distinction.

Same for the tickets. We offer all the chairs in the corresponding auditorium (at a certain time slot, not the actual physical chairs) for sale, but it does not mean that anybody is going to pay for them. Not only is it possible that the movie is boring, but it's also possible that we have to give somebody a ticket for free. Maybe they have a membership card, maybe we owed a replacement ticket.

We discussed all that so that we could answer a simple but important question: how do we name the anchor? (Disclaimer: the author of this book is not a native English speaker. (But this problem exists for me even in my native language.)) What is the short word for an **offer** to sit in a certain chair during a movie

screening? When a customer accepts this offer, they get a ticket. "Ticket" is a nice word, but was there a ticket before the ticket was bought?

Frankly I do not have a good word for that, maybe we could just use "Ticket"? We could distinguish between tickets sold and tickets left unsold by just having a yes/no attribute. This model may be somewhat confusing for some people: you have a table called "tickets" that has many more records than the number of actual tickets sold.

Another problem is that, as we said before, it may not even be "sold": you can get a ticket without paying for it. We need to find some verb for that, maybe we could use "assigned"? Let's try writing it down.

Anchors:

Anchor	*ID example*	*Physical table name*
Ticket	1, 2, 3, ...	—

And attributes:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Ticket	Was the Ticket **assigned**?		
yes/no	*"yes"*	—	
Ticket	What is the **unique number** of the Ticket?		
string, unique	*"164552746"*	—	

Tickets refer to a Time slot and to a Chair:

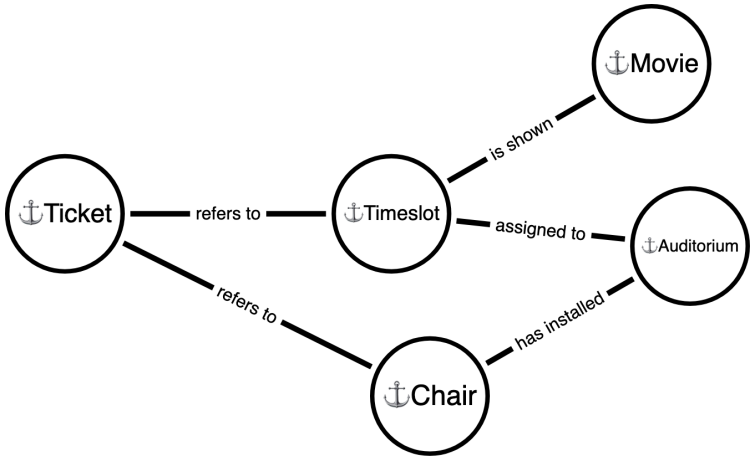
Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Timeslot : Ticket	**1:N**	A Time slot can have *several* Tickets A Ticket can refer to *only one* Time slot	—
Chair : Ticket	**1:N**	A Ticket can refer to *only one* Chair A Chair can be used for *several* Tickets	—

Here we used the most bland verb of them all: "to have". You must resist the temptation to use this verb whenever possible, because it would creep in and half of the links in your logical model would be "this has that" and "those have this". In data modeling, the only verb worse than "to have" is "to be", and we'll see some examples in the following chapter.

Note that the second link is defined as "*A Ticket can refer to only one Chair*". Is it so? In the cinemas that I've been to, every guest receives a separate ticket. But it's easy to imagine a slightly different arrangement (maybe a different business) where a reservation includes more than one seat (or something else). Which one is right? That's for you to decide.

10.7 A DIAGRAM

For illustration, here is a diagram that shows all anchors and links for the cinema use case. You can compare it to the graph of departments presented above and see that it is roughly similar.



10.8 CONCLUSION

In this chapter we've learned a "repeated sales" pattern, using movie tickets as an example.

Also, we learned how to split a large business domain into departments that depend on each other. We saw how it helps to model the database area by area, step by step building the complete database schema.

We discussed some gotchas in the informal language that may confuse you during modeling.

I can offer you two exercises to try after reading this chapter. First is to model the areas that we omitted, for example how to deal with payments.

Second exercise would be to model a different business that uses a "repeated sales" pattern, for example plane tickets. There are at least three different meanings of the word "flight" there, can you find them all and choose a good name for the corresponding anchor?

11

BOOKS AND WASHING MACHINES: POLYMORPHIC DATA PATTERN

11.1 BUSINESS REQUIREMENTS

Imagine an e-commerce website that sells a lot of different things. For example, books and washing machines, but also clothes, bicycles, LED lamps, vinyl records, groceries, gift cards, etc., etc.

For each type of item we want to help users find the best item that we offer. For example, people shopping for a washing machine may want to search by the vendor name, capacity, physical dimensions, and for some unique features that the washing machine industry has invented.

People buying books want to search by the author name, cover type, publication date, etc. This is an entirely different set of properties. (Note that we do not say "different set of attributes", because author information requires links too. This is something that is often omitted in discussions of this pattern.)

Same for the bicycles, groceries and so on.

But there are some things that are common between books, washing machines and so on: they can all be ordered and they have a price. You can place a single order for two items: a book and a washing machine, and this order will be delivered to you, maybe in two shipments.

Let's apply our approach to this sort of business and see how it works. Here, in addition to the logical model, we will also extensively discuss the physical table design.

11.2 PER-DEPARTMENT MODELING

Like in the previous chapter, let's think about how we can divide the company into departments.

We are quite serious about selling, so we employ people who are experts in books, experts in washing machines, and so on. So, we have a department per each item type:

- books department;
- washing machines department;
- bicycles department;
- etc.

In addition, we have an order fulfillment department. Those people do not really care about specific items, they want to know only physical dimensions, and where to find the item in the warehouse. They would deliver anything, given an item catalog number and the shipment address.

Another department is responsible for payments. They want to know the price of each item. If a customer paid the price, the order would be approved for shipment.

11.3 KEY INSIGHT: GENERIC ANCHOR VS SPECIFIC ANCHORS

So, if we would look from the point of view of the books department, we would obviously introduce a *Book* anchor. Same for the washing machines department: a *Washing Machine* anchor. But from the point of view of the order fulfillment department, all of those things are some sort of more generic sellable item.

If this department was a separate company that just managed order delivery, they would have just defined an *Item* anchor. This anchor would have attributes such as "weight of an Item", "catalog number of an Item", and other generic information.

To align those two points of view, we could define a link for each of the specific anchors that connects them with a generic *Item* anchor.

Anchors:

Anchor	*ID example*	*Physical table name*
Item	1, 2, 3, ...	⟨TBD⟩
Book	1, 2, 3, ...	⟨TBD⟩
Washing Machine	1, 2, 3, ...	⟨TBD⟩

(Many more specific anchors would also be defined here.)
Attributes:

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Item	What is the **price** of an Item? (in USD)		
monetary amount	*12.99*	⟨TBD⟩	
Item	What is the **weight** of an Item (in kilograms)		
number	*4.5*	⟨TBD⟩	
Book	What is the **title** of the Book?		
string	*"On the Origin of Species"*	⟨TBD⟩	
Washing Machine	What is the **energy label** of the Washing Machine?		
string	*"A++"*	⟨TBD⟩	

(Many more specific attributes of specific anchors would also be defined here, such as the number of pages of the book, or the number of gears of a bicycle). Also, more generic attributes of Item could be defined here, such as a catalog number or physical dimensions).

Links (note that we use a relatively rare 1:1 cardinality here):

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Item : Book	**1:1**	An Item can be *only one* Book A Book can be *only one* Item	⟨TBD⟩
Item : Washing Machine	**1:1**	An Item can be *only one* Washing Machine A Washing Machine can be *only one* Item	⟨TBD⟩

(For each specific anchor, we would add a link between it and the Item. Additionally, for some specific anchors we may add specific links, for example "*Book : Author*".)

A small note on natural language ambiguity is needed here. What does "*Our shop sells 200 books*" mean? Is it 200 different book titles, or 200 copies of the same book? Informally, it's clear that the *Book* anchor corresponds to the title, that is, book considered as a publication. But we would probably prefer to say "*Our shop sells 50 washing machine models*", because "*Our shop sells 50 washing machines*" just sounds weird.

You can be more pedantic in naming anchors, if you want, and choose a *Book Title* and a *Washing Machine Model* as anchor names. In this chapter we would be less precise, let's see if it would bite us later.

11.4 MULTIPLEXING ON ITEM TYPE

If we have a Book, it's clear which Item it corresponds to. We just query the link table using a Book ID to find the Item ID.

But if we have an Item ID, how do we find if it's a book or a washing machine? In practice we would just introduce a simple book-keeping attribute: item type.

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Item	What is the **type** of an Item?		
either/or/or	*"book"* *"washing_machine"* *"..."*	⟨TBD⟩	

We call this "a book-keeping attribute" because it's a rare kind of attribute that is closer to the physical model than usual. We'll discuss this shortly.

Its main purpose is to help you find which specific anchor corresponds to this Item. If you know that the type of item ID=123 is "bicycle" then you can go and retrieve the information that is specific to bicycles (maybe from the bicycle-specific tables).

11.5 TANGLED LINKS

This attribute has another purpose. The description that follows would be more theoretical, but I am not sure if the rest of this section would confuse you or clarify anything.

So, if you think about the links that we defined (*Item : Book*, *Item : WashingMachine*, etc.), you may notice that nothing prevents you from erroneously setting up *both* of those links. An *Item* can be only one *Book*, but the logical schema does not prevent it from also being a *Washing Machine*.

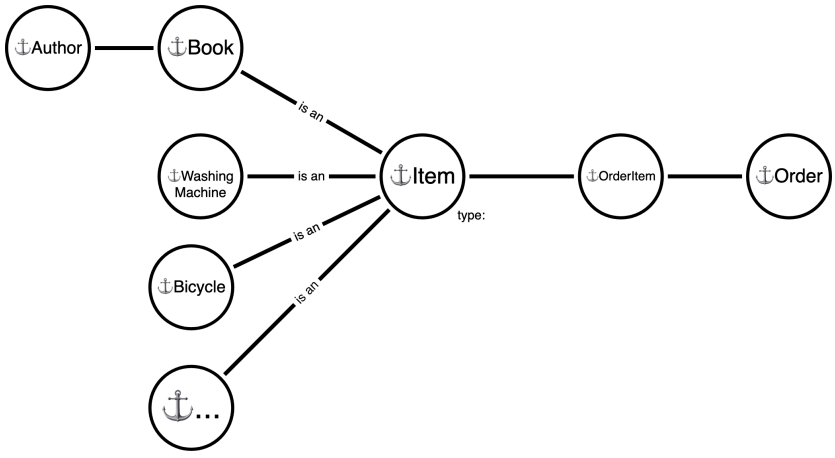
To prevent this, we've introduced the item type, but we also need to introduce a concept of tangled links. It is needed mostly for theoretical completeness. Let's repeat the list of links here, but add one qualifier in the first column:

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Item : Book (for Item type *"book"*)	**1:1**	An Item can be *only one* Book A Book can be *only one* Item	⟨TBD⟩
Item : Washing Machine (for Item type *"washing_machine"*)	**1:1**	An Item can be *only one* Washing Machine A Washing Machine can be *only one* Item	⟨TBD⟩

Here we declare that the link is only valid for items with a certain value of "type" attribute. In the following sections we'll discuss the physical design of this logical schema.

11.6 A DIAGRAM

Here is a graphical overview of the logical mode we have so far:



11.7 TABLE-PER-ANCHOR APPROACH

Technically speaking, we could use the table-per-anchor approach. The design would be very straightforward, with the following tables:

- `items`;
- `order_items`;
- `orders`;
- `books` (and `authors`);
- `waching_machines`;
- `bicycles`;
- etc.

In practice, it seems that almost nobody does that. Let's revisit the four table design concerns and try to find out possible reasons why.

Completeness of the logical schema implementation: perfect.

Complexity of adding a new anchor, link or attribute: you need to create a new table or add a new field every single time — this seems to be problematic for many people. Instead, people prefer approaches that are called flexible, we'll discuss it shortly.

Handling **read-only queries**: we can't go into much detail here. Theoretically, a table-per-everything gives maximum information to the query optimizer. There would be two main classes of queries:

- generic queries, involving only items and orders, without any product-specific information;
- product-specific queries, involving a product table and the items table; we'd use those, for example, to render the invoice.

If we would have a books-only e-commerce system, we'd be perfectly fine with those kinds of queries. But as more and more types of items are added, people become more averse to doing queries that way, and prefer more combined approaches. It's definitely cognitively easier to have a single table "`items`" that contains all the information.

Let's discuss a polymorphic table design strategy, and see how it contrasts with the classic old-school table design.

11.8 POLYMORPHIC TABLE DESIGN STRATEGY

We are fine with product-agnostic tables, such as orders, order_items and items. What we don't want to do is to create and maintain tables per item type, such as books and bicycles.

11.8.1 *JSON-based columns*

Virtually all modern databases support JSON-typed table columns. JSON supports what is called "objects" (name-value pairs). Names are strings, and values can be strings, numbers and booleans (among others). Arrays are also supported. This is enough to store most of the logical types that we have in the logical model.

Here is an example of a book record:

```
{
  "title": "On the Origin of the Species",
  "author_ids": [1035],
  "publication_year": 1859,
  "number_of_pages": 502
}
```

(Note that we have used an array for the list of author IDs, here the list contains just one element. This ID refers to some other table, see the discussion below.)

And here is an example record about of a washing machine:

```
{
  "vendor_name": "LG",
  "model_number": "FH4U2VCN2",
  "energy_label": "A+++"
}
```

11.8.2 *Physical schema*

Here is how we would define the "`items`" table, using a JSON-based column:

```
CREATE TABLE items (
  id INTEGER NOT NULL PRIMARY KEY AUTO_INCREMENT,
  type VARCHAR(24) NOT NULL,
```

```
  price DECIMAL(10,2) NULL,
  weight DECIMAL(10, 4) NULL,
  item_data JSON NOT NULL DEFAULT ('{}')
);
```

Item type is stored in the "`type`" column, and contains a string such as *"washing_machine"*.

We've added two item-agnostic attributes that we've defined for the Item anchor: price and weight.

Also, we've added a JSON-typed column called item_data. It would contain JSON objects such as the one presented above.

There are some important things to note about this design. First, you may notice that the "`item_data`" column is not mentioned as a logical attribute of Item anchor. This is correct: `item_data` lives on a physical level. It can contain many attributes of many anchors, and that's exactly the idea. We'll discuss how this design would be documented in the logical schema shortly.

Second, the JSON objects that we've shown have their own schema: the list of allowed keys and their data types. To store the book title, you must put it into a "`title`" JSON key, and this key only makes sense for items that have `type="book"`. This schema could be documented in the logical model, or maintained separately in the code, for example using the JSON Schema definition file.

You can also use both and try to keep them in sync with each other.

11.8.3 *Storing links in JSON*

You may notice that the following key is actually a link, and not an attribute:

```
{
  "author_ids": [1035]
}
```

One-element array means that the book has exactly one author. You can also have books without authors:

```
{
```

```
  "title": "Code Cenannensis",
  "author_ids": []
}
```

And books with two or more authors:

```
{
  "title": "The Practice of Programming",
  "author_ids": [92608, 517764]
}
```

That array contains a list of IDs. But where would the author information be stored? The easiest answer is that you would most probably just have another table called "`authors`".

11.8.4 *Table design concerns, revisited*

What did we get from this? With this table design, we no longer need to make any schema changes when we add more product types and attributes of existing product types. We just need to agree on the value of the "type" column (such as "washing_machine"), and on the name of the JSON key name (such as "`energy_label`").

We only need to do some changes if we define a new product-specific anchor, such as Author. This should happen relatively rarely.

The "`items`" table becomes almost self-contained. In many environments, the tables get imported into many different data processing tools. Having just one table simplifies operations.

Overall this reduction of organizational friction is worth it for many teams in many environments, and this sort of polymorphic table design seems to be a commonly used strategy. The goal of this chapter is to give you some familiarity with this design, and explain the primary motivation behind it.

11.8.5 *Documenting physical storage*

Now let's revisit the logical model that we defined above, and fill in the information about physical columns.

Anchor	*ID example*	*Physical table name*
Item	1, 2, 3, ...	`items`
Book	*1, 2, 3, ..., same as Item*	`items` (where `type="book"`)
Washing Machine	*1, 2, 3, ..., same as Item*	`items` (where `type="washing_machine"`)
Author	1, 2, 3, ...	`authors`

Here all the item types are stored in the same "`items`" table. We also specify for the human reader how to distinguish different item types.

Information about the authors is stored in a separate table. Its design is straightforward, so we don't discuss it here.

Anchor	*Question*	*Column name (physical)*	*Data type (physical)*
Data type	*Example value*		
Item	What is the **price** of an Item? (in USD)		
monetary amount	*12.99*	`items.price`	`DECIMAL(10, 2)`
Item	What is the **weight** of an Item (in kilograms)		
number	*4.5*	`items.weight`	`DECIMAL(10, 4)`
Book	What is the **title** of the Book?		
string	*"On the Origin of Species"*	`items.item_data: $.title`	*JSON string*

Anchor *Data type*	*Question* *Example value*	*Column name (physical)*	*Data type (physical)*
Washing Machine	What is the **energy label** of the Washing Machine?		
string	*"A++"*	`items.item_data: $.energy_label` *JSON string*	
Item	What is the **type** of an Item?		
either/or/or	*"book"* *"washing_machine"* *"..."*	`items.type` `VARCHAR(24)`	

Some attributes here are contained in the normal table columns, such as "`items.price`". Product-specific attributes are documented using the JSON Path notation:

`items.item_data: $.title`

The physical type is specified as "JSON string".

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Item : Book	**1:1**	An Item can be *only one* Book A Book can be *only one* Item	`items.id`
Item : Washing Machine	**1:1**	An Item can be *only one* Washing Machine A Washing Machine can be *only one* Item	`items.id`

Anchor1 : Anchor2	*Cardinality*	*Sentences*	*Table or column name (physical)*
Book : Author	**M:N**	A Book can have *several* Authors An Author can write *several* Books	`items.item_data: $.author_ids[]`

Here the 1:1 links are actually contained in a single column: `items.id`. That's because, for example, a Book with ID=13 is the same as an Item with ID=13.

Links that are stored in JSONs are documented using JSON Path notation:

```
items.item_data: $.author_ids[]
```

We don't have fully standardized notation, we just want it to be concise and understandable to people. You can add extra human-readable explanations in this column if you need.

12

PRACTICALITIES

You can experiment with the approach presented in this book using any tool that you're familiar with. You need to maintain three tables: list of anchors, list of attributes and list of links. Also, you need a place to keep a textual description of your project.

You also need to maintain the physical database schema somewhere. It depends on the way you develop your system. It may be an SQL file in your source repository, or an ORM definition, or maybe you directly design the tables in your database using some sort of UI, graphical or text-based. It's possible that you're only interested in the logical schema so you skip this part.

12.1 DOCUMENT-BASED CATALOG

It seems that the most natural choice here is some word processor, such as Google Docs (that's what I use). Tables in this book are formatted for readability, due to demands of e-book readers. For the actual table design process you need something simple.

On the following page you can see some example Google Docs screenshots (before typesetting).

12.2 SPREADSHEET-BASED CATALOG

Another type of tool that you could use is a spreadsheet, for example Google Sheets.

Create three tabs: anchors, attributes and links. Input the column names, and make the header "frozen", so that it stays on the screen when you scroll the rest of the document. Enable text wrapping on all the columns, and set text alignment to top.

Google Docs screenshots

(a) Anchors

Noun	ID example	Physical table
User	*1, 2, 3, …*	`users`
Show	*1, 2, 3, …*	`shows`
Episode	*1, 2, 3, …*	`episodes`

(b) Attributes

Anchor	Question	Data type	Example value	Physical column	Physical type
User	What is the User's **email address**?	string, unique	*"somebody@example.com"*	`users.email`	`VARCHAR(64) NOT NULL`
User	What is the User's encrypted **password**?	string	*"$2a$12$R9h/cIPz0gi."*	`users.password`	`VARCHAR(128) NOT NULL`
Show	What is the **name** of the Show?	string	*"Complex Systems with Patrick McKenzie"*	`shows.name`	`VARCHAR(192) NOT NULL`
Episode	What is this Episode's **cover image**?	binary blob	*[JPEG file contents]*	*<TBD>*	*<TBD>*
Episode	What is this Episode's **air date**?	date	*2025-01-16*	`episodes.air_date`	`DATE NULL`

(c) Links

Anchor1 : Anchor2	Cardinality	Sentences	Physical table or column
User : Show	1:N	A User can own *several* Shows A Show can be owned by *only one* User	`shows.owner_user_id`
User : Episode	1:N	A User can upload *several* Episodes An Episode can be uploaded by *only one* User	`episodes.uploader_user_id`
Show : Episode	1:N	A Show includes *several* Episodes An Episode can be included in *only one* Show	`episodes.show_id`

One small problem with spreadsheets is that you would need an extra document to keep the textual description of your project.

You can use paper if you like, or a whiteboard.

12.3 HOW MUCH TO WRITE

The goal of this approach is to help you with the design. The goal of the logical schema is to find and fix mistakes as early as possible: while editing a document.

You can have business requirements in your head, or written down. Writing it down may help you to organize your thoughts somewhat. If you work in a collaborative environment you would certainly want to write down your understanding of the problem. It's also possible that somebody actually wrote down the business requirements for you, and your task is to convert it to a more structured schema, and maybe even build a database.

Every piece of the logical schema has some purpose, a reason for you to write it down in full.

Anchor names could be either trivial or insightful. Trivial names are something like User or Post. Insightful anchor names would clarify complicated situations: see "Repeated sales pattern" for some examples.

Anchor IDs are either trivial or interesting. Trivial IDs use sequential numbers (1, 2, 3, ...). Some examples of interesting anchor IDs could be found in the "Well-known anchors" chapter.

Anchor table names are either straightforward or non-obvious. Straightforward name: "`users`". Non-obvious names arise, for example, in the polymorphic design situations, see "Polymorphic data pattern".

Attribute anchors are needed mostly to filter the list of attributes by anchors. It's useful to have an overview of all the attributes that belong to a certain anchor. Otherwise, the name of an anchor is usually repeated in the attribute question.

Attribute questions are a teaching innovation of this book, and I'm proud of them. If you're new to database modeling, writing down the questions in full would help you. As you get more familiar with the database design, you'll find that sometimes asking the right question helps you to see that the attribute should belong to a different anchor (and fix this mistake). At the

same time, I understand that many people would still shorten the questions, maybe down to a single word. Sometimes it works, but if it doesn't you can always try asking the question and see if it helps.

Logical data type of the attribute is important when you work on the logical schema. Maybe you did not even choose the database yet, or you're not going to design the database yourself. We use several logical data types that you should pay particular attention to. They were chosen for a reason. Particularly: yes/no values, either/or/or values, monetary values (as opposed to other numeric types), date/times (UTC as opposed to local). They help you to prevent some common design mistakes early.

Example values of the attribute are essential. Just a single example of an actual data value can immensely help with understanding the database structure. Especially if you're lazy with the attribute questions, do not skip example values. They help you to cross-check the other pieces of data and quickly find inconsistencies (for example, because of copy-paste errors).

Physical columns of the attribute need to be precisely defined. You can skip this if you're not going to design the physical table. Sometimes somebody else would be doing that, and that would be their decision. Specifying a physical column is essential for collaboration. You would avoid a lot of back and forth if you just write down how to get this piece of data.

Physical type of the attribute needs to be precisely defined, otherwise you cannot implement the physical table. As in the previous paragraph, sometimes you're not going to work on the physical design, so you can skip this.

Now let's get to **link anchors**. You need to specify a pair of anchors so that you could quickly find the links that affect a certain anchor. Both names would be repeated in the link sentences. Remember that links require that each pair of anchor IDs is unique; this can help with detecting false links (see the "False links" section).

Link cardinality has its own table column because it helps during the physical table design. Usually, 1:N links belong to the N-side anchor table, and M:N links have their own table. Technically, cardinality is also expressed in the link sentences,

but it's so important that it just has its own place. Also, it is super short.

Link sentences are another teaching innovation of this book. If you write down both sentences and they make sense in the real world then your design is correct. As you get more experienced with the database design, you'll find that sentences help to make sure that the business requirements are correctly implemented. **Link cardinality mistakes are the most expensive to fix.** That is why we go the extra mile to confirm it.

Physical table/column of the link needs to be precisely defined. You can skip this if you're not going to design the physical table. When the table is designed, writing down this information is essential for collaboration: people need to quickly understand where the data is.

Note that you can add **custom columns** to the logical schema document. One example is regulated data, such as personally-identifiable information (PII), financial and health data.

12.4 LIGHTWEIGHT DESIGNS

You don't need to build a full upfront design of the entire system, like they do in the enterprise software environments. The goal of this approach is to help you, not to control you.

Particularly, writing down the business requirements may help, but is not required. We're not doing the waterfall here.

You can design the entire project from scratch, or only the part of the system that you struggle with. If the approach helped and you made a break-through in understanding, you can discard the document. (If it did not, I'd be interested to hear why.)

You do not need to make the schema document permanent. Maybe you're a lone developer and you have no need to collaborate. Maybe you're doing your course assignment (for example, an ERD diagram), or preparing for the interview.

If you're at a job interview or in a database exam you would have to keep everything in your head. Here, anchor names, questions and sentences may help you to double-check the correctness of your schema.

The goal of this approach is to help people internalize the process of database design.